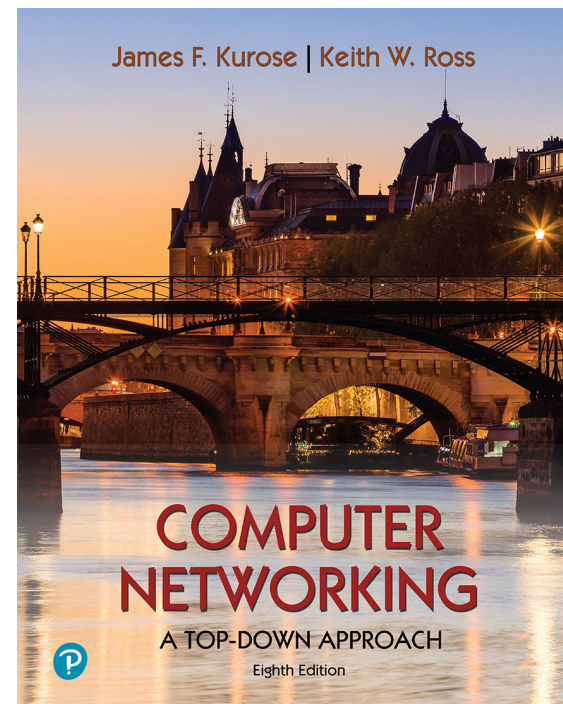


第 4 章

网络层：数据平面

中国科学技术大学
自动化系 郑烱
改编自 Jim kurose, Keith Ross



*Computer Networking: A
Top-Down Approach*

8th edition

Jim Kurose, Keith Ross

Pearson, 2020

网络层： 目标

- 理解网络层服务背后的原理，聚焦于数据平面：
 - 网络层服务模型
 - 转发 vs 路由
 - 路由器结构和工作原理
 - 编址
 - 通用转发
 - 互联网体系架构
- 互联网网络层-数据平面的实例和实现
 - IP协议
 - NAT和中间盒

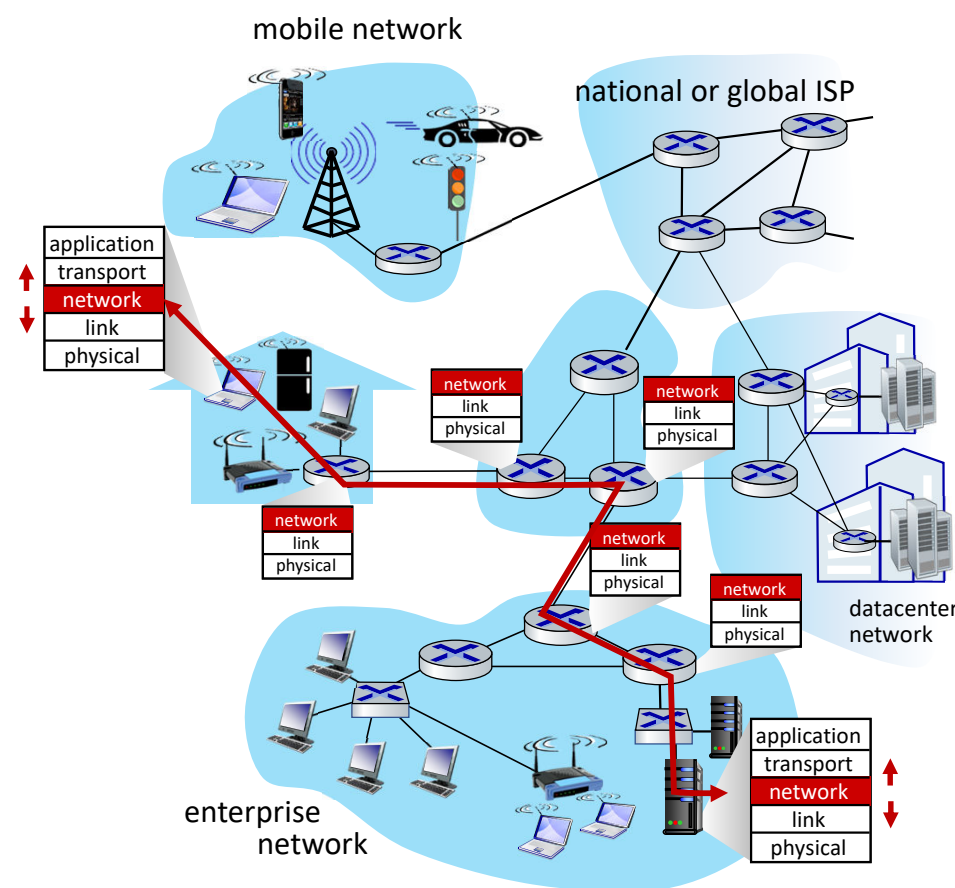
网络层： 之数据平面 提纲

- 网络层： 概述
 - 数据平面
 - 控制平面
- 路由器的结构和工作原理
 - 输入端口，交换机构，输出端口
 - 缓存管理，调度
- IP: the Internet Protocol
 - 数据报格式
 - 编址
 - 网络地址转换： NAT
 - IPv6
- 通用转发，SDN
 - 匹配+行动
 - OpenFlow: 匹配+行动 运行
- 中间盒



网络层服务和协议

- 在发送主机和接收主机对（端到端）之间传送段
 - 发送端：将段封装到数据报中，交给链路层
 - 接收端：将段上交给传输层实体
- 网络层协议存在于每一个互联网设备：主机和路由器
- 路由器：
 - 路由器检查每一个经过它的IP数据报的头部
 - 将数据报从输入端口转到输出端口，将数据报沿着端到端路径，一跳一跳向目标主机传输



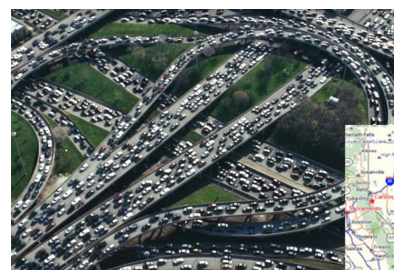
2个网络层关键功能

网络层功能：

- **转发：** 将分组从路由器的输入链路转发到合适的输出链路
- **路由：** 使用路由算法来确定分组从发送主机到目标接收主机的路径
 - 路由算法
 - 路由协议

类比：旅行路线

- **转发：** 通过单个立交口的过程
- **路由：** 规划从源到目标路径的规划过程



forwarding



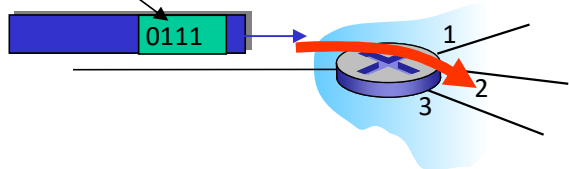
routing

网络层：数据平面、控制平面

数据平面：

- *局部的*，每个路由器的功能
- 决定每个到达路由器的分组如何从输入端口转发到输出端口

values in arriving
packet header

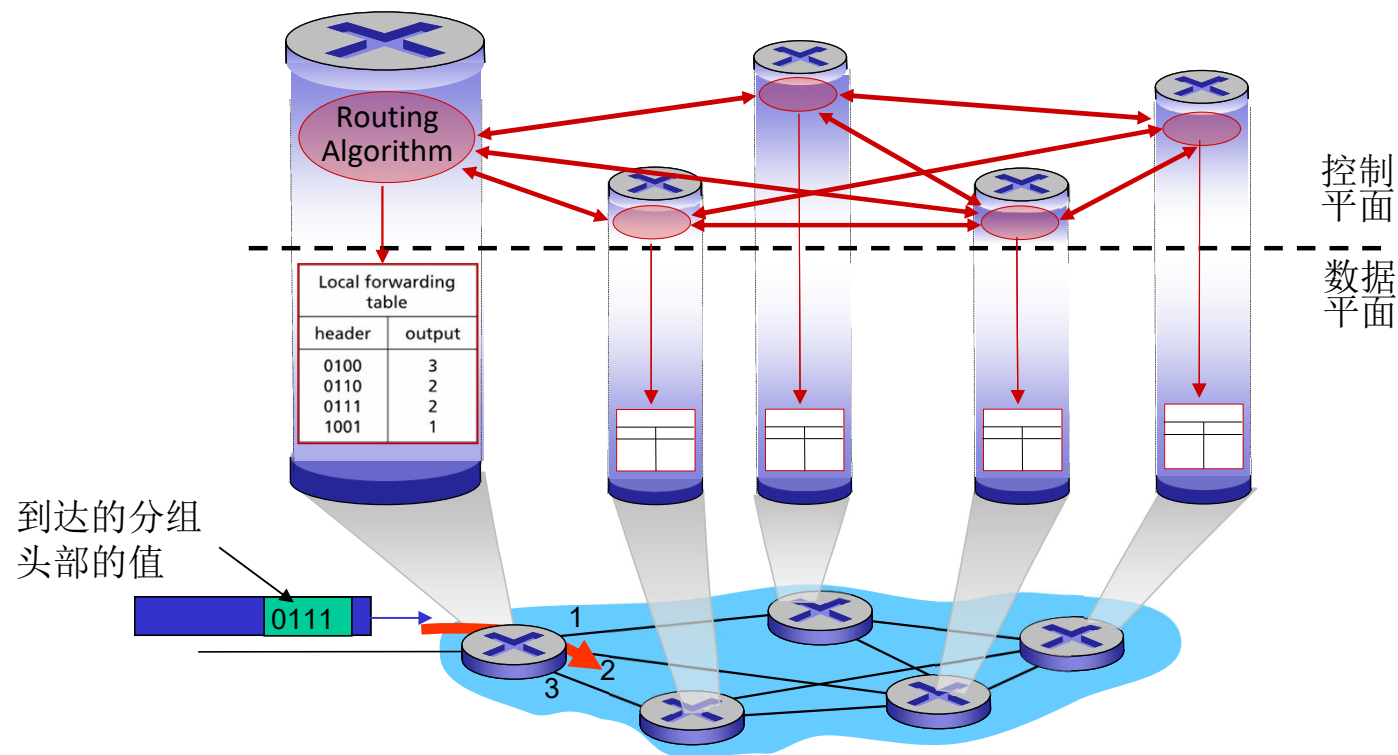


控制平面

- *网络范围*的逻辑
- 决定数据报如何沿着端到端路径从源端到达目标端的路径
- 2个控制平面的方法：
 - *传统路由算法*：在路由器上实现
 - *software-defined networking (SDN)*：在（远程）的服务器上实现

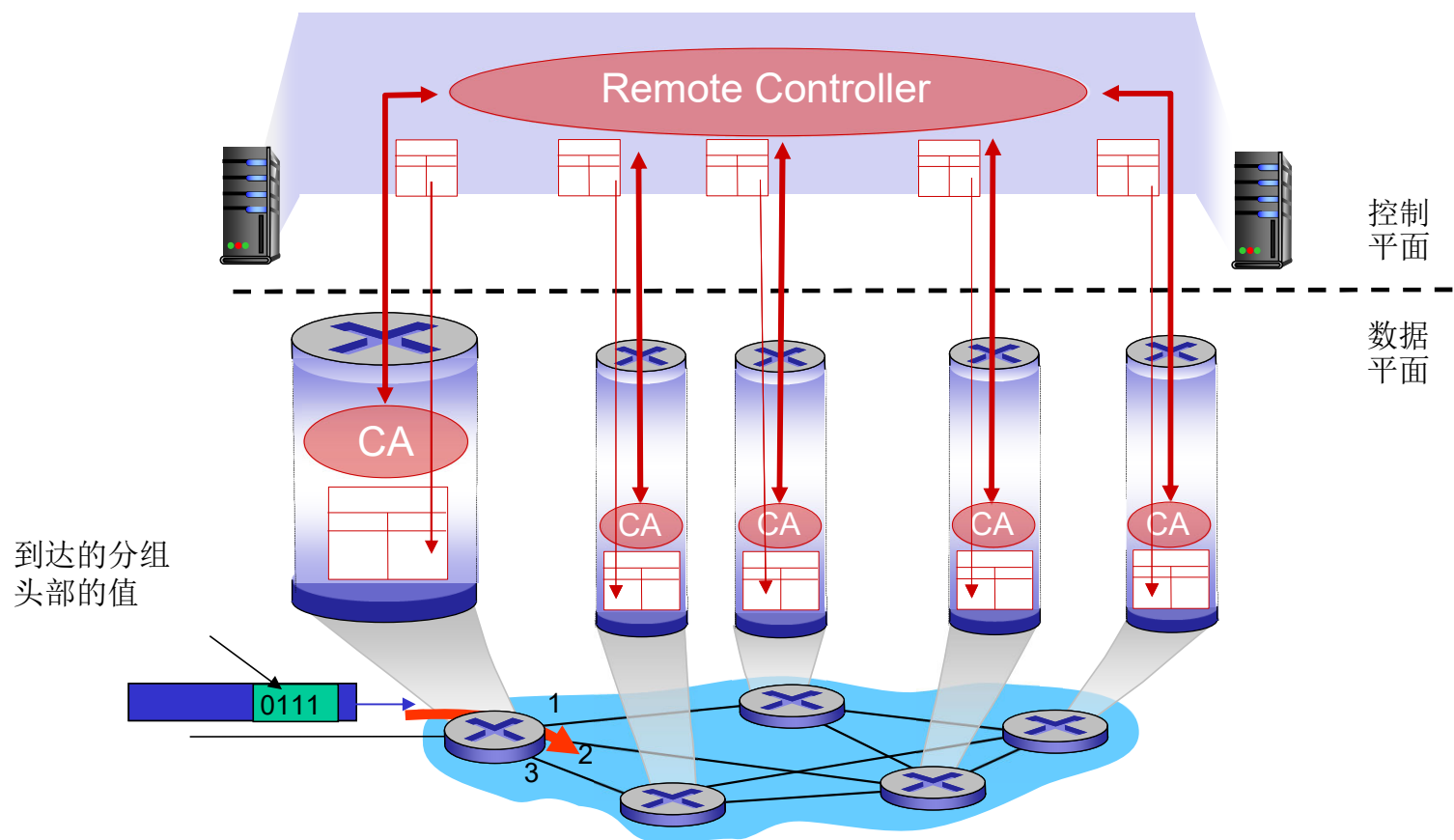
传统方式：每路由器(Per-router) 控制平面

独立的路由算法元件运行在每个路由器中，路由器在控制平面上进行交互



Software-Defined Networking (SDN) 控制平面

由远程的控制器计算并在安装在交换设备上的转发表



网络服务模型

*Q:*将数据报从发送方到接收方“通道”，其服务模型是什么？

针对单个数据报的服务模型例子：

- 保证交付与否
- 如在40ms延迟之内保证交付

针对数据报流的服务模型例子：

- 数据报保序交付
- 能够确保给该流的最小带宽
- 分组之间的间隔变化上限（抖动上限）

网络层服务模型

网络 体系架构	服务 模型	服务质量(QoS: Quality of Service) 保证 ?			
		带宽	丢失	次序	延迟
Internet	best effort	none	no	no	no

Internet “尽力而为” 服务模型

没有对以下指标提供任何保障:

- i. 成功交付到目标主机的数据报
- ii. 延迟以及交付的顺序
- iii. 端到端流的可用带宽

网络层服务模型

网络 体系架构	服务 模型	服务质量(QoS: Quality of Service) 保证 ?			
		带宽	丢失	次序	延迟
Internet	best effort	none	no	no	no
ATM	Constant Bit Rate	Constant rate	yes	yes	yes
ATM	Available Bit Rate	Guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

关于IP层次仅提供尽力服务的思考

- IP提供的简单机制能够让互联网被广泛地部署
 - 能够包容各种异构网络
- 如果提供足够的带宽能够，基本就让实时应用（如：交互式音视频）的性能在“大部分时间”内“足够好”
- 应用层面实现的分布式服务副本（数据中心，内容分发网络）可以让服务更加靠近用户，允许用户从多个位置使用
- 简单的尽力而为的IP数据报服务，滥用会拥塞，端系统上的“弹性”拥塞控制有助于网络的正常运行
 - 网络拥塞，主机减少往网络中注入的速率
 - 网络轻载，主机加快速率

尽力而为服务模式是成功的，这点难以反驳

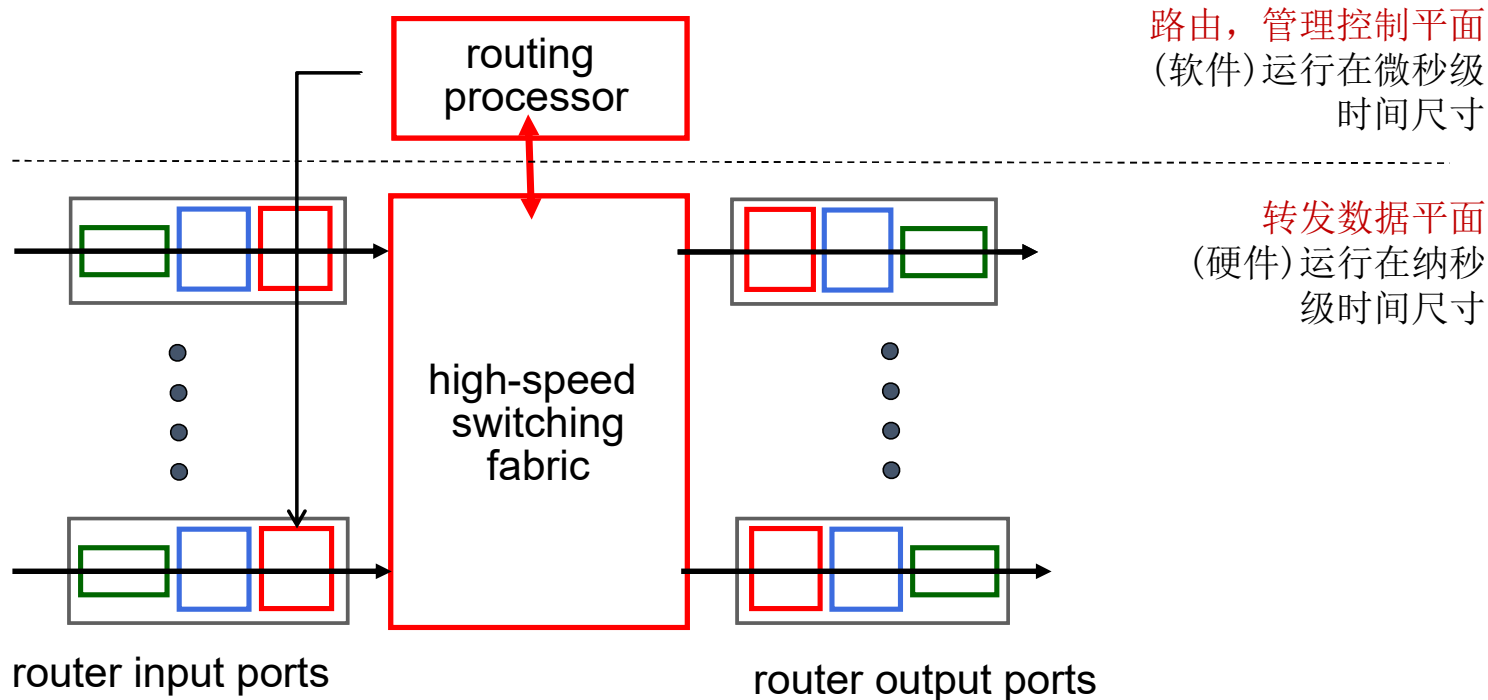
网络层：“数据平面”提纲

- 网络层：概述
 - 数据平面
 - 控制平面
- 路由器的结构和工作原理
 - 输入端口，交换机构，输出端口
 - 缓存管理，调度
- IP: the Internet Protocol
 - 数据报格式
 - 编址
 - 网络地址转换：NAT
 - IPv6
- 通用转发，SDN
 - 匹配+行动
 - OpenFlow: 匹配+行动 运行
- 中间盒

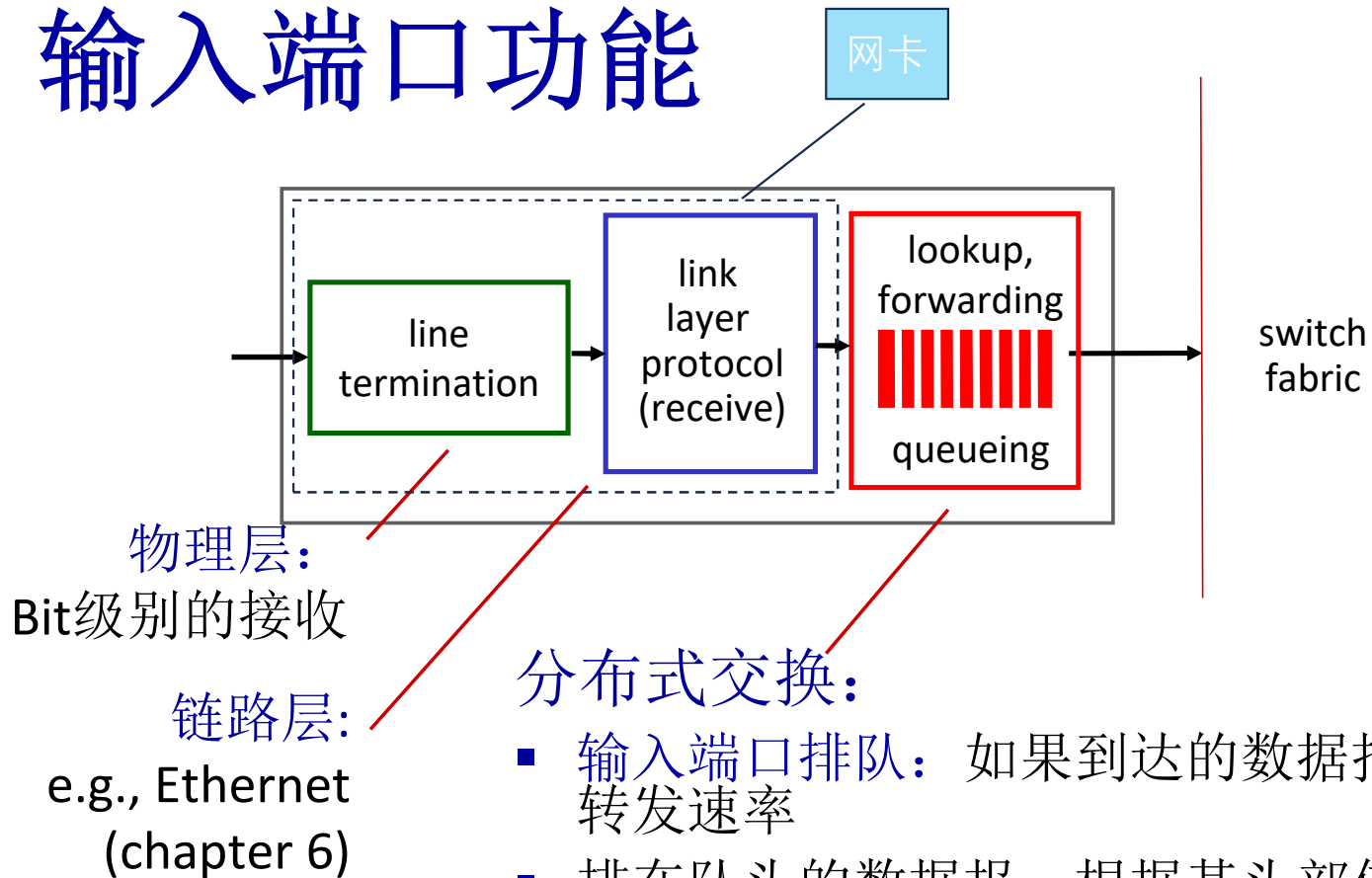


路由器架构概述

通用路由器架构的高层视图



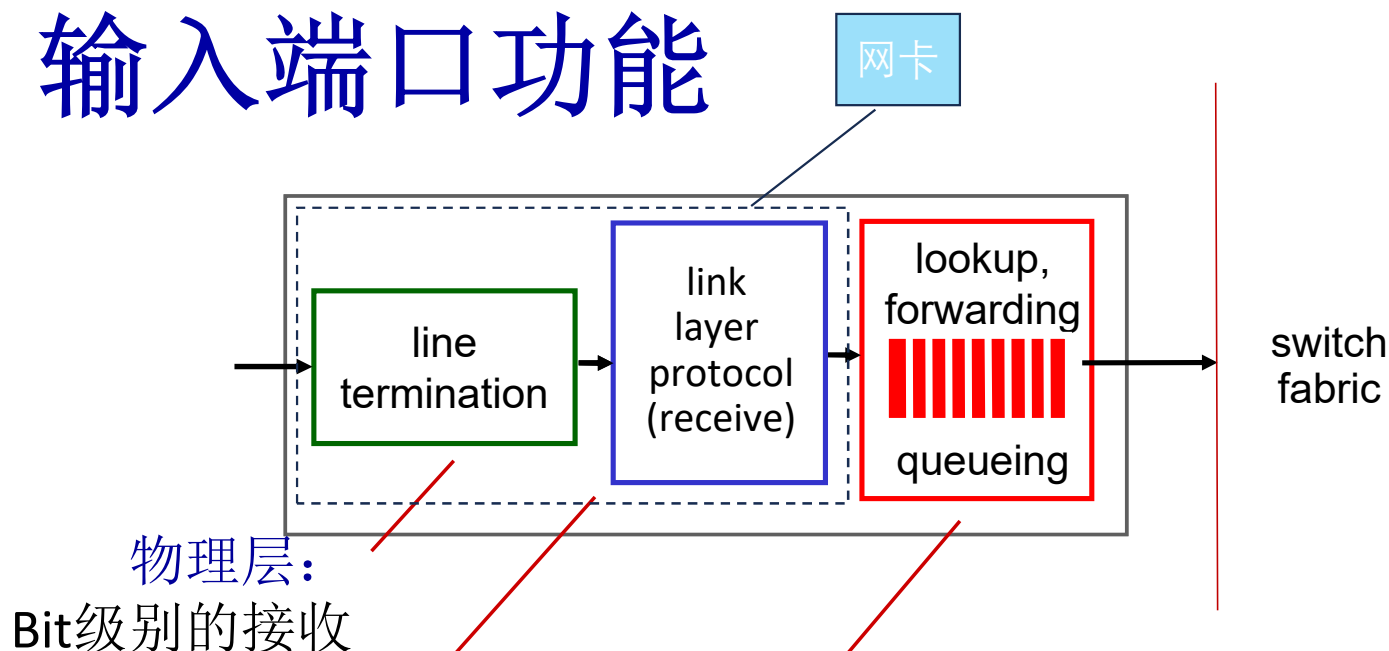
输入端口功能



分布式交换:

- **输入端口排队:** 如果到达的数据报速率快于在交换机构的转发速率
- 排在队头的数据报, 根据其头部信息, 在输入端口内存中的转发表中查找合适的输出端口 (匹配+行动)
- 目标: 以“线速”完成输入端口的处理

输入端口功能



物理层:

Bit级别的接收

链路层:

e.g., Ethernet
(chapter 6)

分布式交换:

- 根据数据报头部的信息，在输入端口内存中的转发表中查找合适的输出端口（匹配+行动）
- **基于目标的转发:** 仅仅依赖于IP数据报的目标IP地址（传统方法）进行转发
- **通用转发:** 基于头部的任意字段集合进行转发或者其他动作

基于目标地址范围进行转发

地址范围	接口	下一跳
11001000 00010111 00010000 00000000 ~ 11001000 00010111 00010111 11111111	0	IPx
11001000 00010111 00011000 00000000 ~ 11001000 00010111 00011000 11111111	1	lpy
11001000 00010111 00011000 00000000 ~ 11001000 00010111 00011111 11111111	2	lpz
otherwise	3	IPx

基于目标地址范围进行转发

地址范围的规律：网络号特定，主机号从全0到全1
(与地址规划相配合就能够达到规整的效果)

地址范围	接口	下一跳
11001000 00010111 00010000 00000000 11001000 00010111 00010111 11111111	0	IPx
11001000 00010111 00011000 00000000 11001000 00010111 00011000 11111111	1	lpy
11001000 00010111 00011000 00000000 11001000 00010111 00011000 11111111	2	lpz
otherwise	3	IPx

基于目标地址范围进行转发

地址范围的简洁表示：子网号/子网掩码->匹配地址范围的快速性

子网号	子网掩码	接口	下一跳
11001000 00010111 00010000 00000000	11111111 11111111 11111000 00000000	0	IPx
11001000 00010111 00011000 00000000	11111111 11111111 11111111 00000000	1	lpy
11001000 00010111 00011000 00000000	11111111 11111111 11111000 00000000	2	lpz
otherwise		3	IPx

基于目标地址范围进行转发

目标子网号	子网掩码	接口	下一跳
11001000 00010111 00010000 00000000	11111111 11111111 11111000 00000000	0	IPx
11001000 00010111 00011000 00000000	11111111 11111111 11111111 00000000	1	lpy
11001000 00010111 00011000 00000000	11111111 11111111 11111000 00000000	2	lpz
otherwise		3	IPx

❑ 路由器读取IP数据报的目标地址

❑ 对于转发表中的每一个表项

✓ 如 $(IP\ Des\ addr) \& (mask) == destination$, 则按照表项对应的接口转发该数据报

✓ 如果都没有找到, 则使用默认表项转发数据报

最长前缀匹配

longest prefix match

给定目标IP地址, 在转发表中查找匹配表项时, 使用**最长前缀匹配**。

Destination Address Range	Link interface
11001000 00010111 00010 *** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

examples:

11001000	00010111	00010 110	10100001	which interface?
11001000	00010111	00011000	10101010	which interface?

最长前缀匹配

longest prefix match

给定目标IP地址, 在转发表中查找匹配表项时, 使用**最长前缀匹配**。

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 match! 1 00011*** *****	2
otherwise	3

examples:

11001000 00010111 00010110 10100001 which interface?
 11001000 00010111 00011000 10101010 which interface?

最长前缀匹配

longest prefix match

给定目标IP地址, 在转发表中查找匹配表项时, 使用**最长前缀匹配**。

Destination Address Range				Link interface
11001000	00010111	00010***	*****	0
11001000	00010111	00011000	*****	1
11001000	00010111	00011***	*****	2
otherwise				3

match!

examples:

11001000	00010111	00010110	10100001	which interface?
11001000	00010111	00011000	10101010	which interface?

最长前缀匹配

longest prefix match

给定目标IP地址, 在转发表中查找匹配表项时, 使用**最长前缀匹配**。

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

match!

examples:

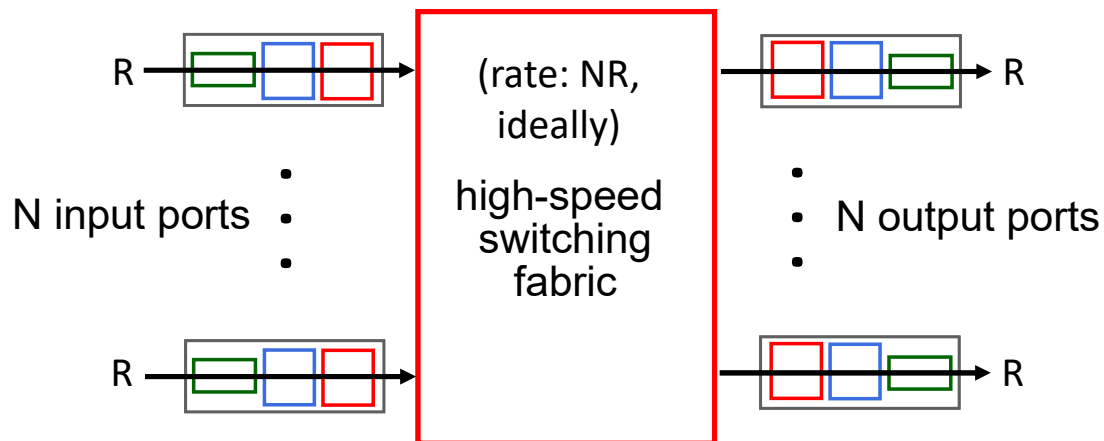
11001000 00010111 00010110 10100001	which interface?
11001000 00010111 00011000 10101010	which interface?

最长前缀匹配

- 在后续学习IP编址时，会理解为什么采用最长前缀匹配
- 最长前缀匹配：在路由器中常采用三态内容可寻址存储器 (TCAM: ternary content addressable memories) 硬件进行
 - **内容可寻址**：将IP地址交给TCAM，它可以在一个时钟周期内检索出匹配表项，不管表空间有多大
 - Cisco Catalyst系列路由器：在TCAM中可以存储多达约为1百万条路由表项

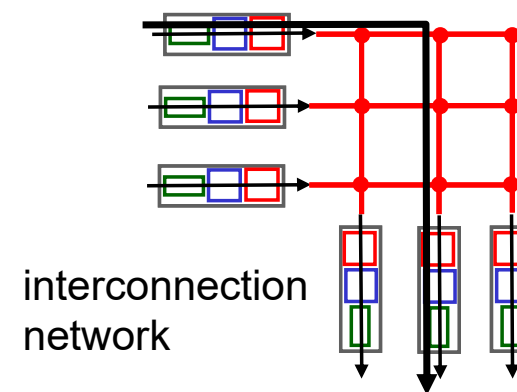
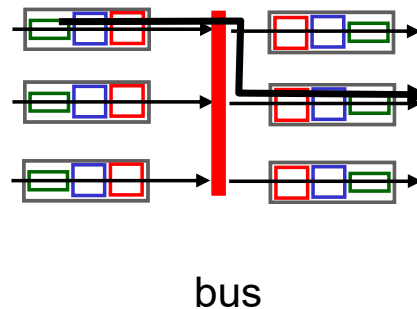
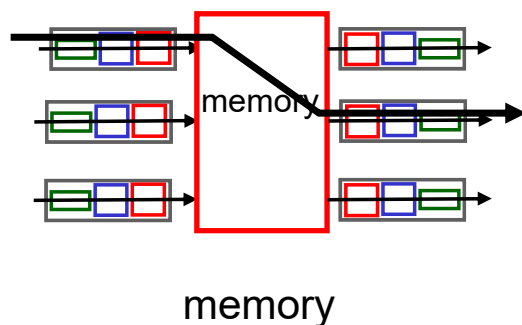
交换结构

- 将分组从输入缓冲区传输到合适的输出端口
- **交换速率**：按照该速率将分组从输入端口转移到输出端口
 - 运行速度经常是输入/输出链路速率的若干倍
 - **N个输入**：交换机构的交换速度是输入线路速度的**N**倍比较理想，才不会成为瓶颈



交换结构

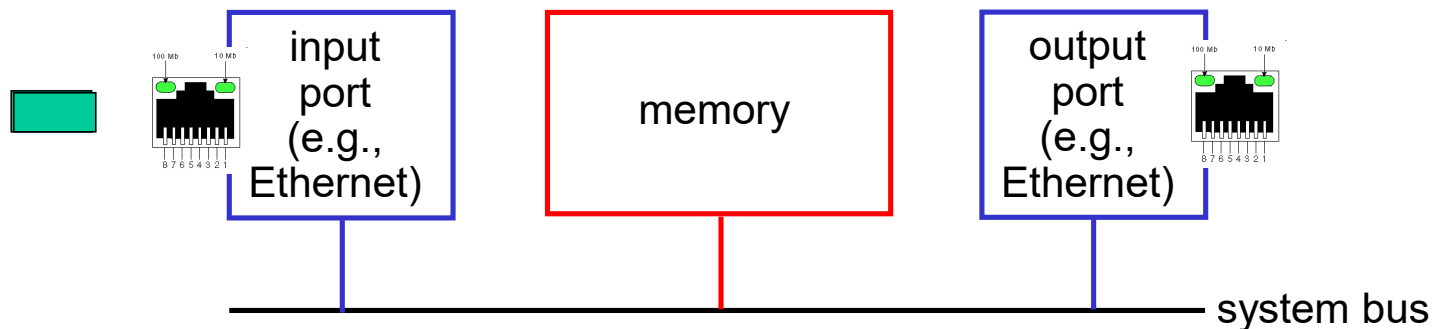
- 将分组从输入缓冲区传输到合适的输出端口
 - ❖ **交换速率**：分组可以按照该速率从输入传输到输出
 - 运行速度经常是输入/输出链路速率的若干倍
 - N 个输入端口：交换机构的交换速度是输入线路速度的 N 倍比较理想，才不会成为瓶颈
- 3种典型的交换机构：



通过内存交换

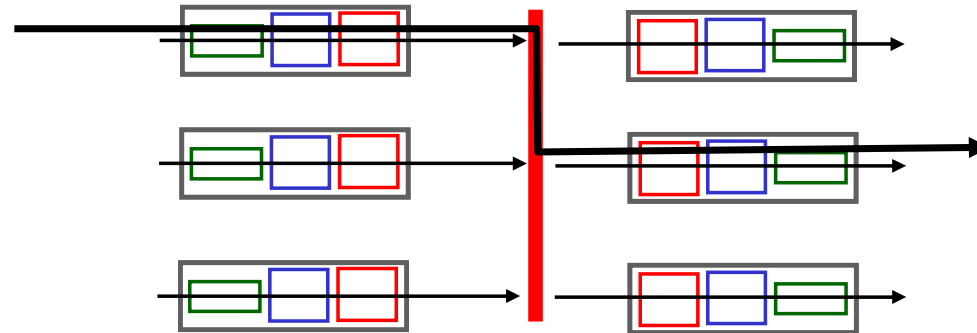
第一代路由器:

- 在CPU直接控制下的交换，采用传统的计算机
- 分组被拷贝到系统内存，计算机CPU从分组的头部提取出目标IP地址，查找转发表，找到匹配表项所对应的输出端口，拷贝到端口
- 转发速率被内存的带宽所限制 (数据报通过BUS两遍)



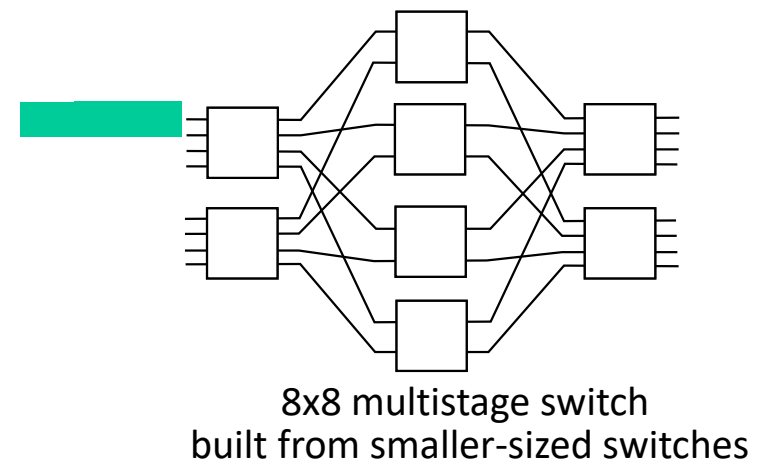
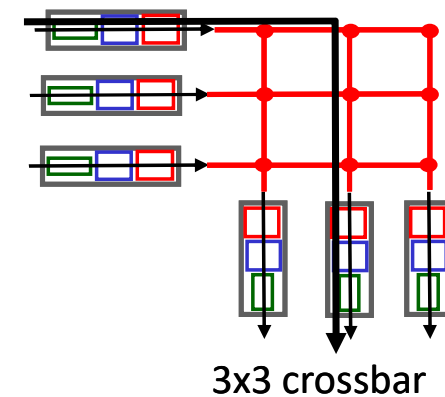
通过总线交换

- 通过共享总线，数据报从输入端口转发到输出端口
- 总线竞争: 交换速度受限于总线带宽
- 32 Gbps bus, Cisco 5600: 该方案对于接入路由器速率足够了



通过互联网络(interconnection network)交换

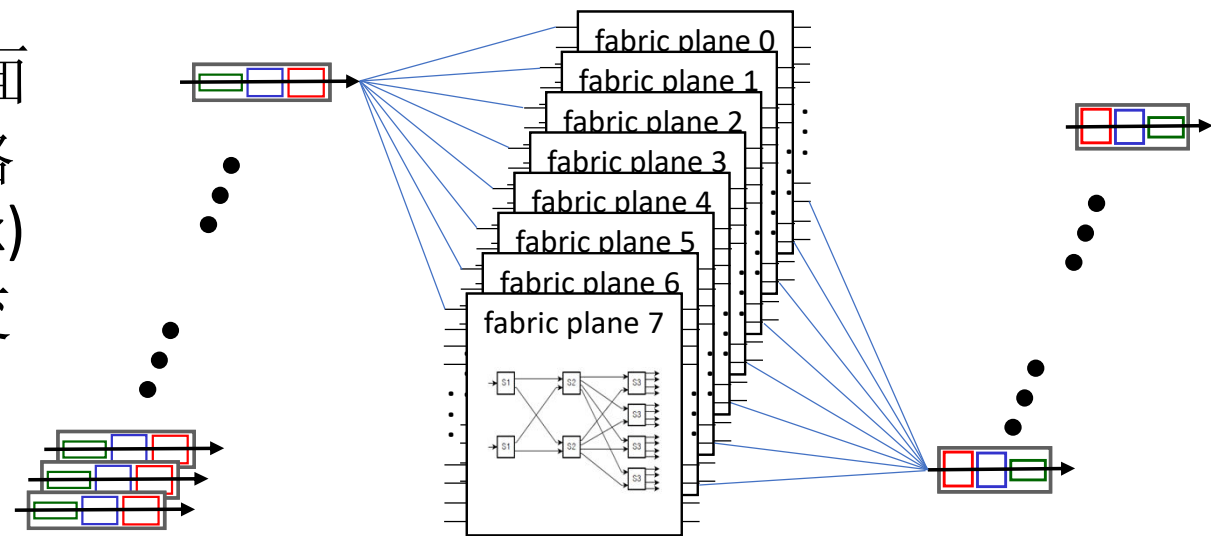
- Crossbar、Clos网络和其他 interconnection nets 技术，最初开发是用于计算机连接多个处理器
- 多级交换：来自于多级小型交换机技术的 $N \times N$ 交换
- 利用并行性：
 - 入口将数据报打成固定长度的cell
 - 交换cell通过交换机构后，在出口处重组该数据报



通过互联网络(interconnection network)交换

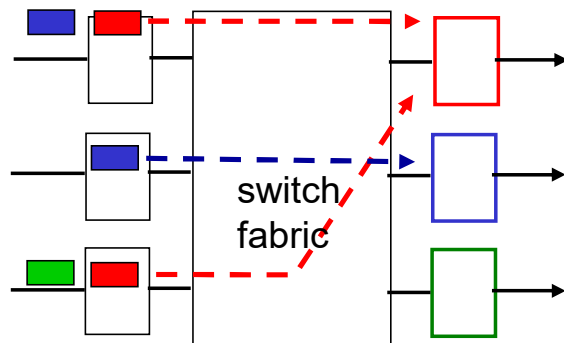
- 进一步扩展：采用多个并行的交换“平面”：
 - 通过并行化进行加速、扩展

- Cisco CRS 路由器：
 - 基本单元：8 个交换平面
 - 每个平面：3级互联网络 (interconnection network)
 - 最大可达到100Tbps的交换能力

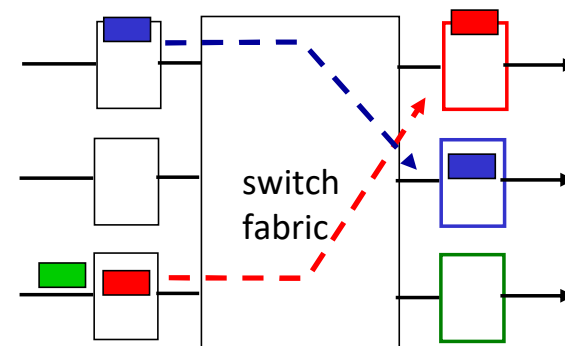


输入端口排队

- 如果交换机构的速率慢于从输入端口进来的组合速率->在输入队列上就会发生排队
 - 出现排队延迟，极端情况输入缓冲可能溢出造成分组丢失
- **Head-of-the-Line (HOL) 阻塞**: 在队头上排队的数据报被阻塞，阻止后面数据报在队列中往前移动

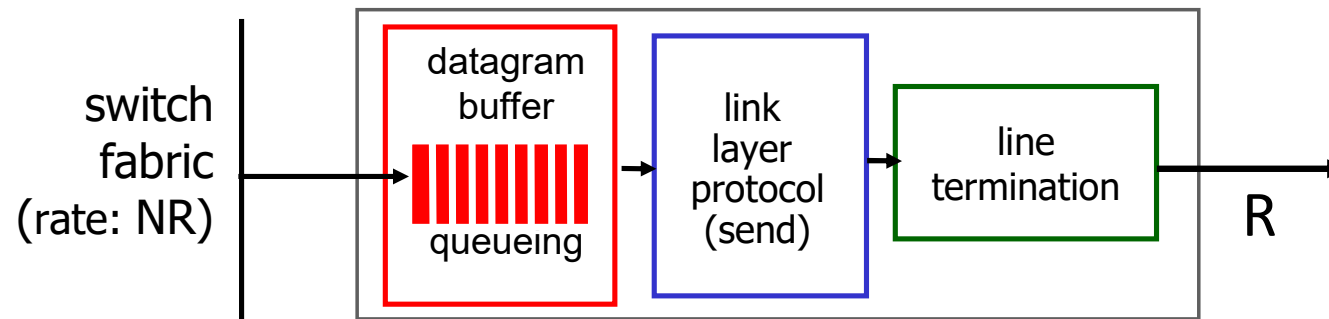


输出端口竞争:只能有一个红色分组被传递，交换到一个输出端口，下面红色的分组被**阻塞**



一个分组时间:
绿色分组经历了头端阻塞

输出端口排队



这是一张相当重要的幻灯片

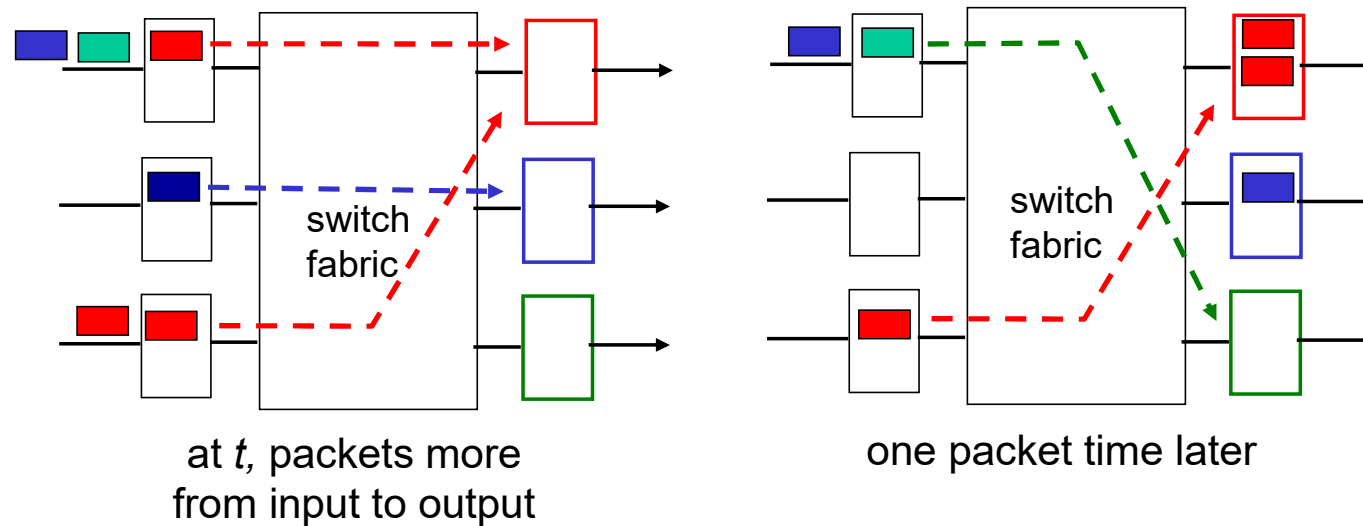
- **缓冲:** 从fabric到来的分组速率快于输出链路传输速率, 因此需要缓冲。
丢弃策略: 在没有缓冲空间时, 丢弃哪个数据报?

数据报可能由于拥塞, 没有缓存空间而丢失

- **调度策略:** 选择哪个正在排队的数据报进行传输

优先级调度-哪种数据报会获得最优性能, 网络中立性

输出端口排队



- 通过交换机构到来的数据报速率通常会超过链路的传输速率
- 排队会产生延迟，极端情况下输出端口缓存会用完造成数据报丢失!

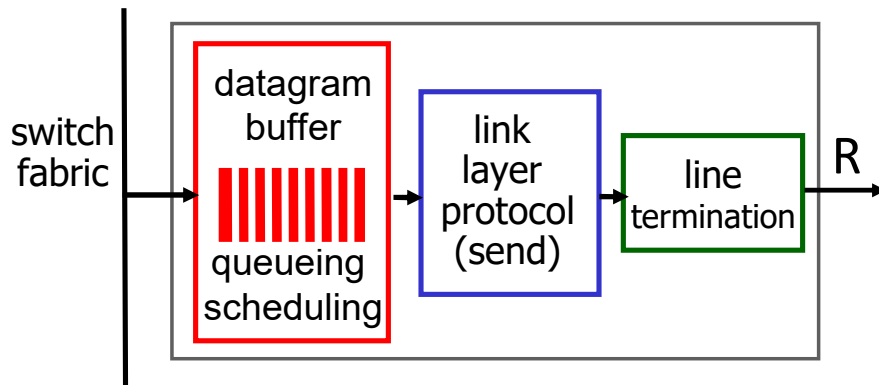
需要多少缓存?

- RFC 3439 拇指规则（经验性规则）：平均缓存大小=典型的RTT（例如：250ms）乘以链路带宽C
 - e.g., C = 10 Gbps link: 2.5 Gbit 缓冲
- 最近的一些推荐：有N个流，缓存大小等于

$$\frac{RTT \ C}{\sqrt{N}}$$

- 太大的缓冲可以减少丢失，但是会增加延迟（特别是家庭路由器）
 - 长的RTT：使得TCP响应延迟大（测得和计算出的RTO较大），对于实时应用来说性能比较差
 - 回想一下基于延迟的拥塞控制：“保持瓶颈链路处在足够满（忙碌）的状态，但是不要太满—否则会造成延迟太大、影响应用效果，也可能使源端超时带来重发进一步加重拥塞”

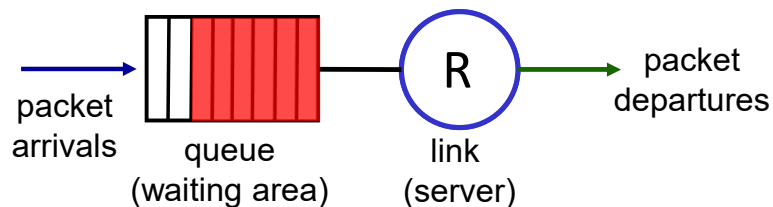
缓存管理



缓存管理:

- **丢弃**: 当缓存满需要增加分组时, 队内的哪个分组将会被丢失
- **优先级**: 基于优先级丢弃/移除数据报

Abstraction: queue



- **标记**: 哪个分组将会被标记来传递拥塞信息 (ECN, RED)

分组调度：FCFS

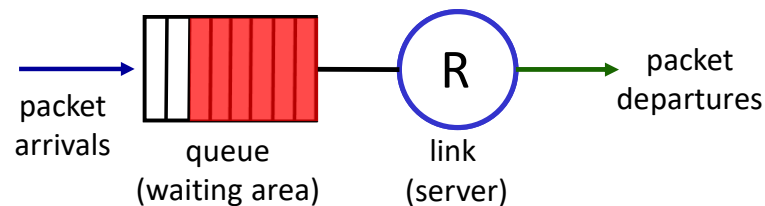
分组调度： 决定下一步哪个分组将会在链路上传输

- FCFS: first come, first served
- 优先权
- RR: round robin
- WEQ: weighted fair queueing

FCFS: 分组传输的次序等同于分组到来的次序

- 也被称为： First-in-first-out (FIFO)
- 现实世界中的例子？

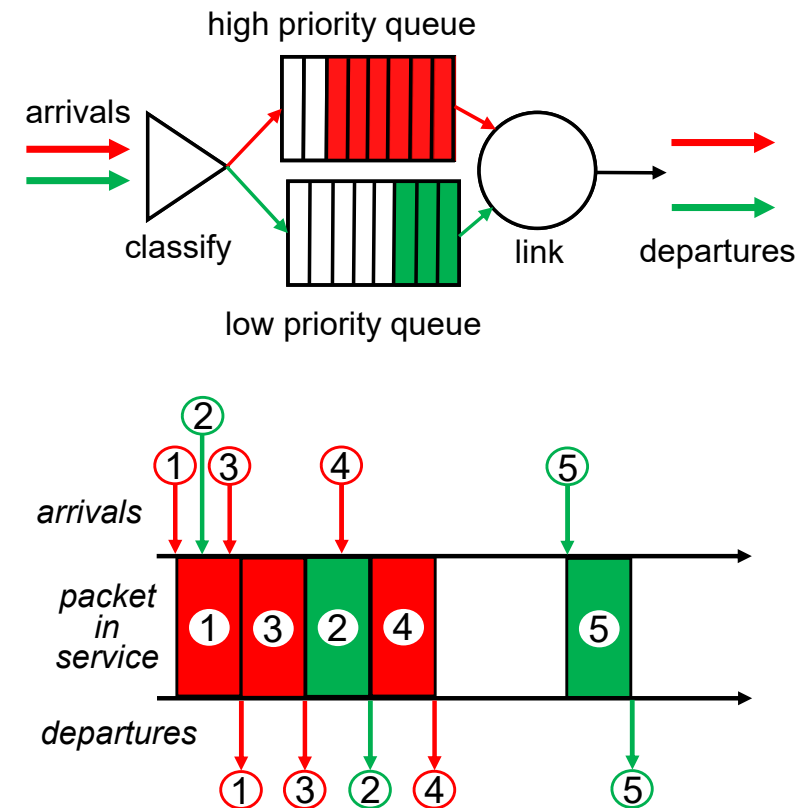
抽象：队列



调度策略：优先权

优先权调度:

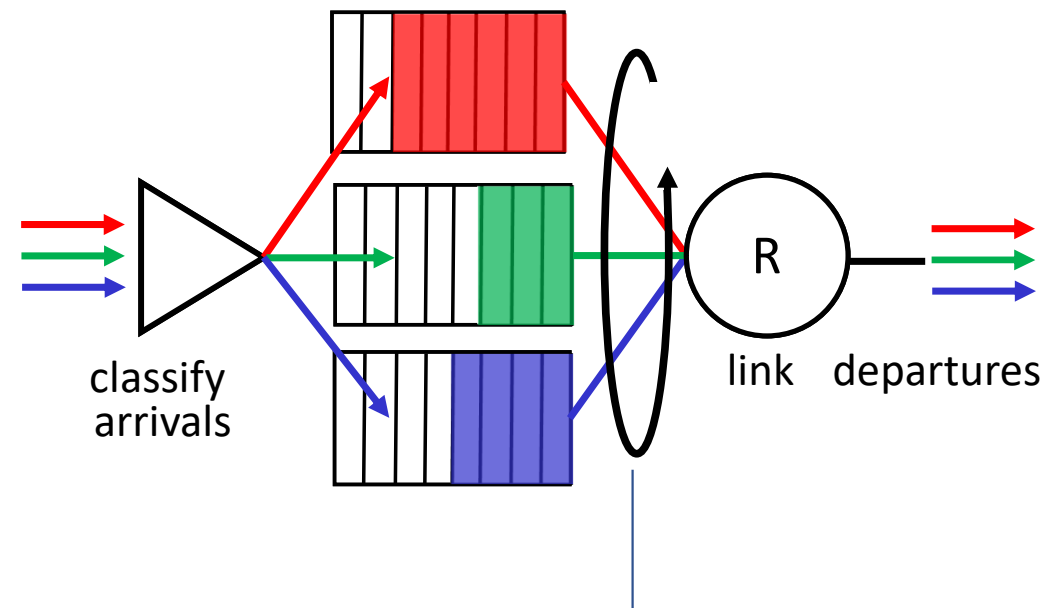
- 到来分组是被分类的，按照类别排队
 - 任何头部字段可以被用于分类
- 从当前有分组的、最高优先权的队列中拿出分组进行传输
 - 在同一个优先权队列内采用 FCFS



调度策略：Round Robin

Round Robin (RR) 调度：

- 到来的分组是被分类的，排队在不同类型的队列中
 - 任何头部字段可以被用于分类
- 服务器循环、重复地扫描各类队列，依次从每个类（如果有分组）发送一个完整的数据报



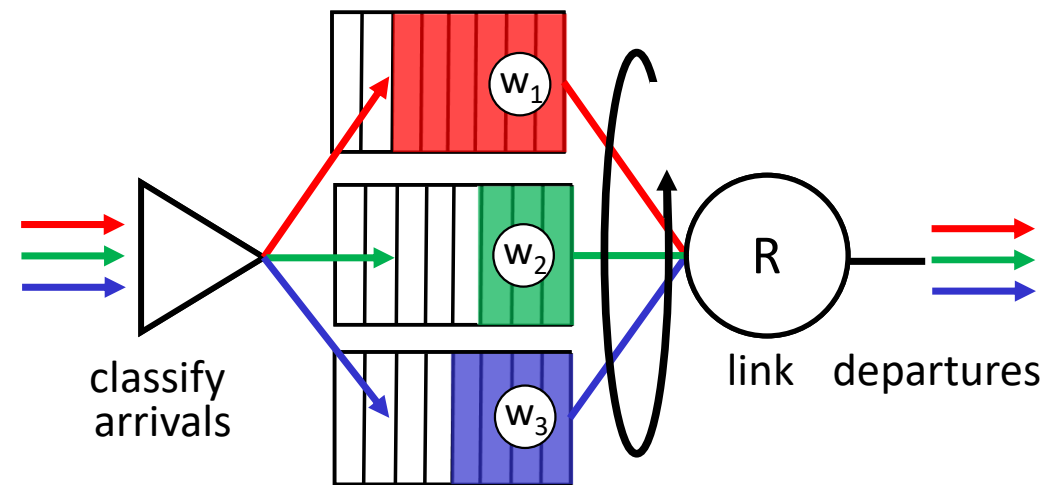
调度策略：加权公平队列

Weighted Fair Queuing (WFQ):

- 一般化的Round Robin
- 类型*i*分组拥有权重 w_i ，在每个循环周期获得加权之后的服务量

$$\frac{w_i}{\sum_j w_j}$$

- 对于每一类能够获得最低带宽保障



Sidebar: 网络中立性

什么是网络中立性？

- *技术上*：对于一个ISP来说他应该如何共享/分配它的资源
 - 分组调度、队列管理（丢弃）是具体的技术机制
- *社会，经济*原则：
 - *保护言论自由*
 - 鼓励创新和竞争
- 强制执行的法律规则和政策

不同的国家对网络中立性有不同的“看法”

Sidebar: 网络中立性

2015年美国FCC关于保护和促进开放互联网的法令：三条“清晰、明亮”规则：

- **不要屏蔽：** ... “在服从合理的网络管理下，不得屏蔽合法的内容、应用程序、服务或无害设备”
- **不遏制流量** ... “在服从合理的网络管理下，不得基于互联网内容、应用程序、服务或使用无害设备，损害或降低合法互联网流量”
- **没有付费优先级**...“不得参与有偿优先级排序”

ISP: 电信或者信息服务?

ISP是电信服务提供商还是信息服务提供商?

- 从监管的角度来看，答案真的很重要！

1934年和1996年《美国电信法》：

- 条款II: 对电信服务征收“共同承运人税”：合理费率，不歧视，需要监管
- 条款I: 适用于信息服务：
 - 无共同承运人税（不受监管）
 - 但授予FCC权力“.....在执行其职能时可能需要”

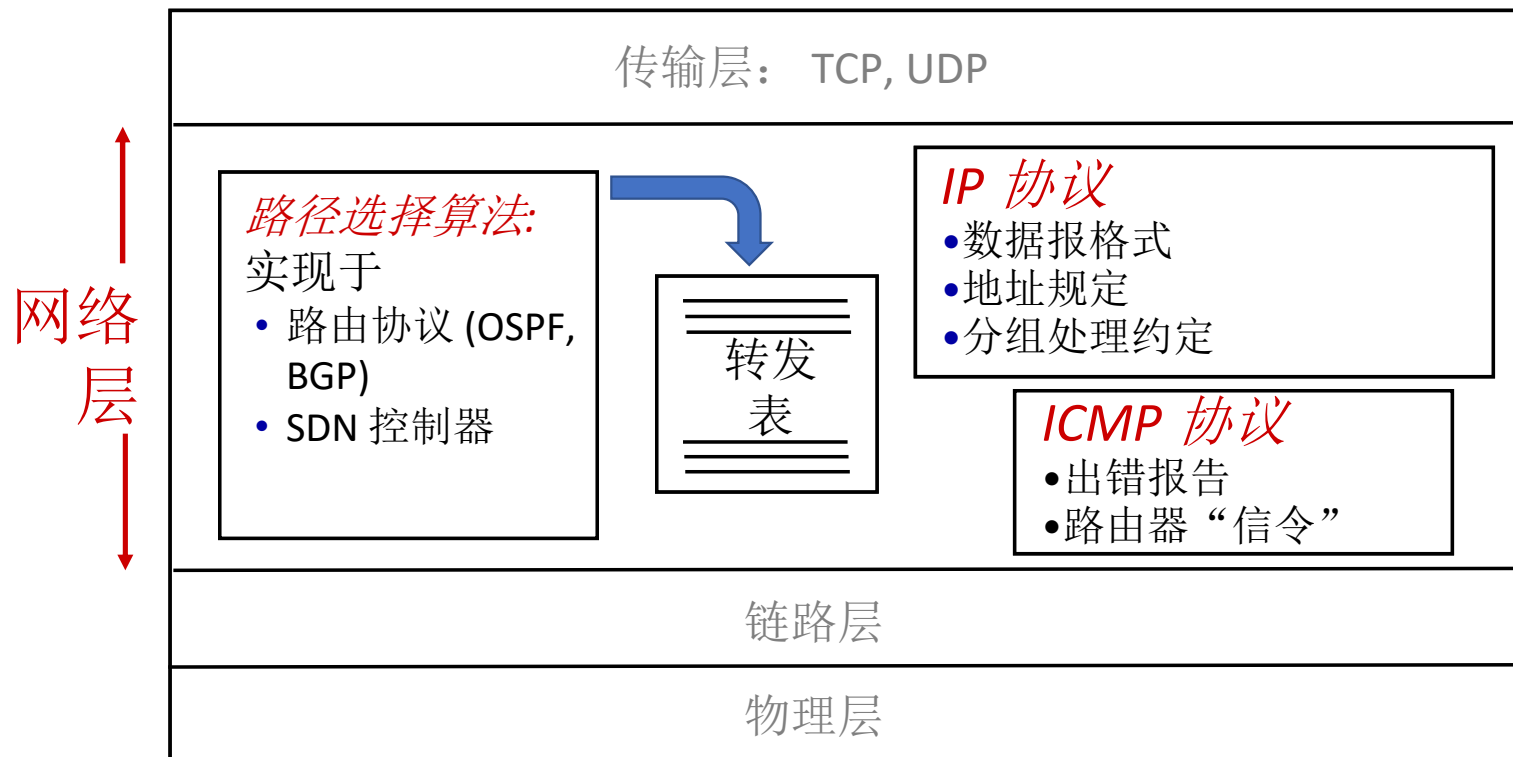
网络层：“数据平面” 提纲

- 网络层：概述
 - 数据平面
 - 控制平面
- 路由器内部结构
 - 输入端口，交换机构，输出端口
 - 缓存管理，调度
- IP: the Internet Protocol
 - 数据报格式
 - 编址
 - 网络地址转换：NAT
 - IPv6
- 通用转发，SDN
 - 匹配+行动
 - OpenFlow: 匹配+行动
- 中间盒

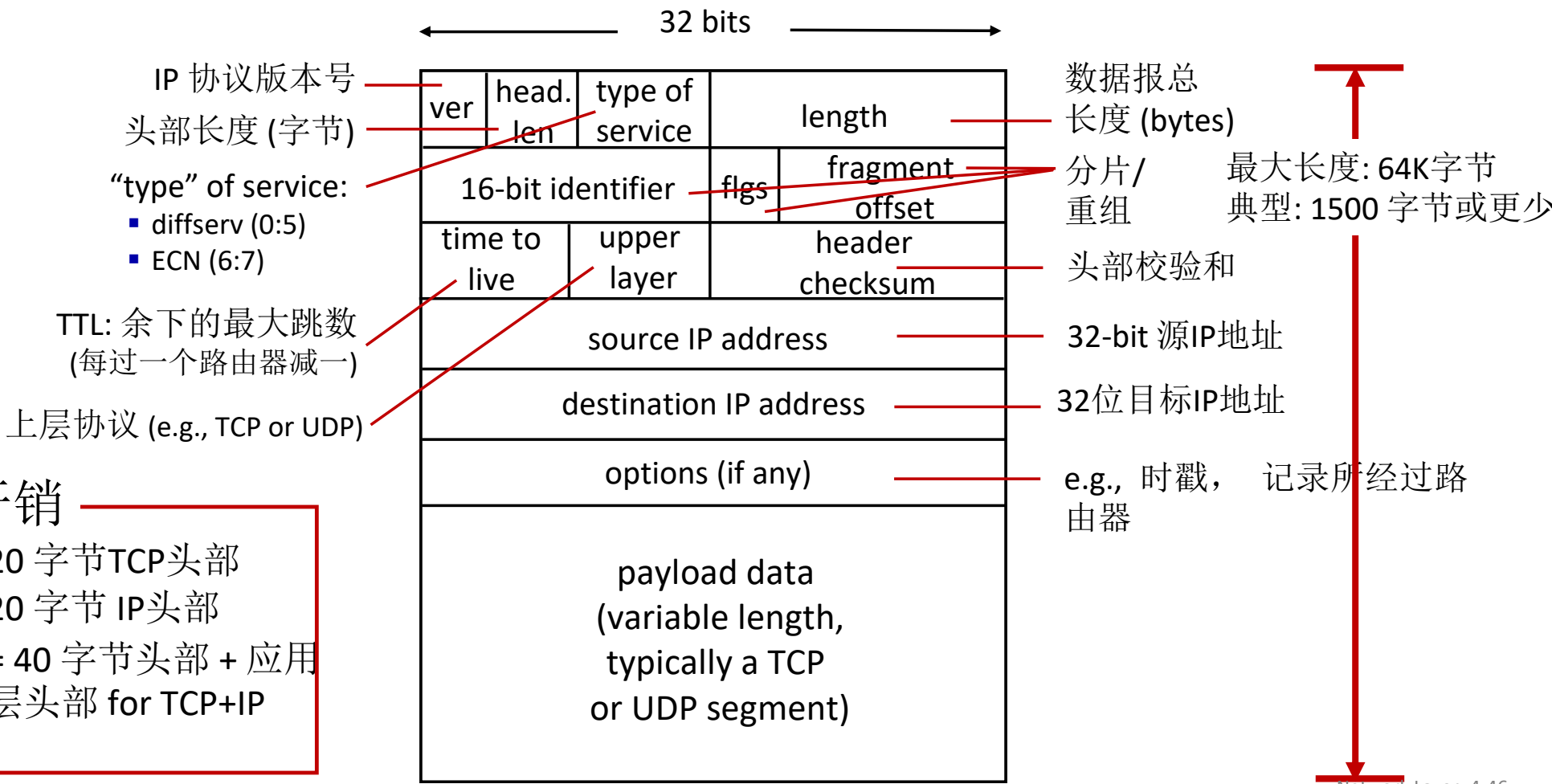


网络层：互联网

主机和路由器的网络层功能:

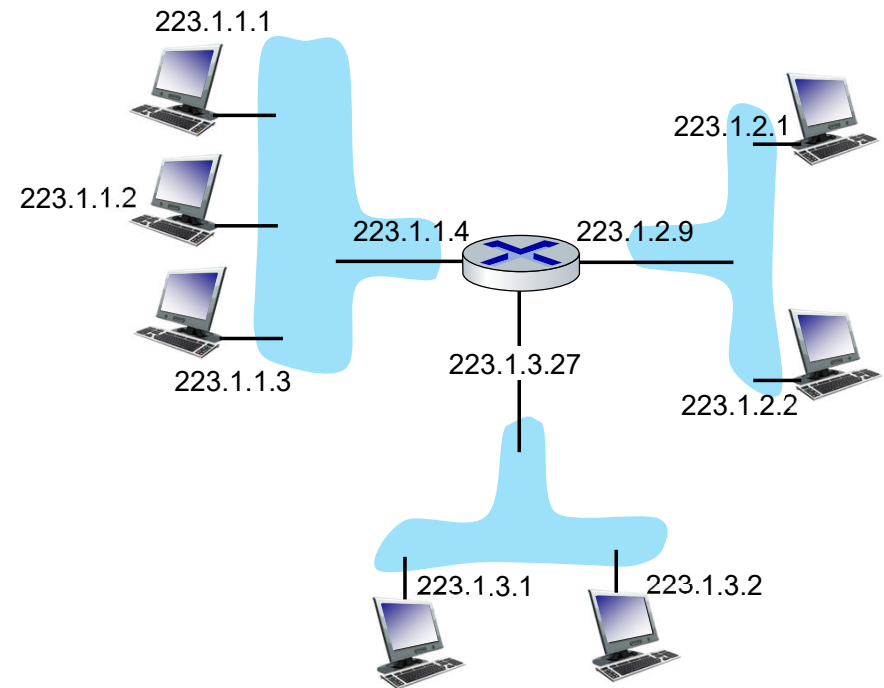


IP 数据报格式



IP 地址: 导论

- **IP 地址**: 32位标示, 对主机或者路由器的**接口**编址
- **接口**: 主机/路由器与物理链路的连接处
 - 路由器通常拥有多个接口
 - 主机典型的有1个或者2个接口 (例如: 有线以太网, 无线802.11) 也有可能拥有多个接口



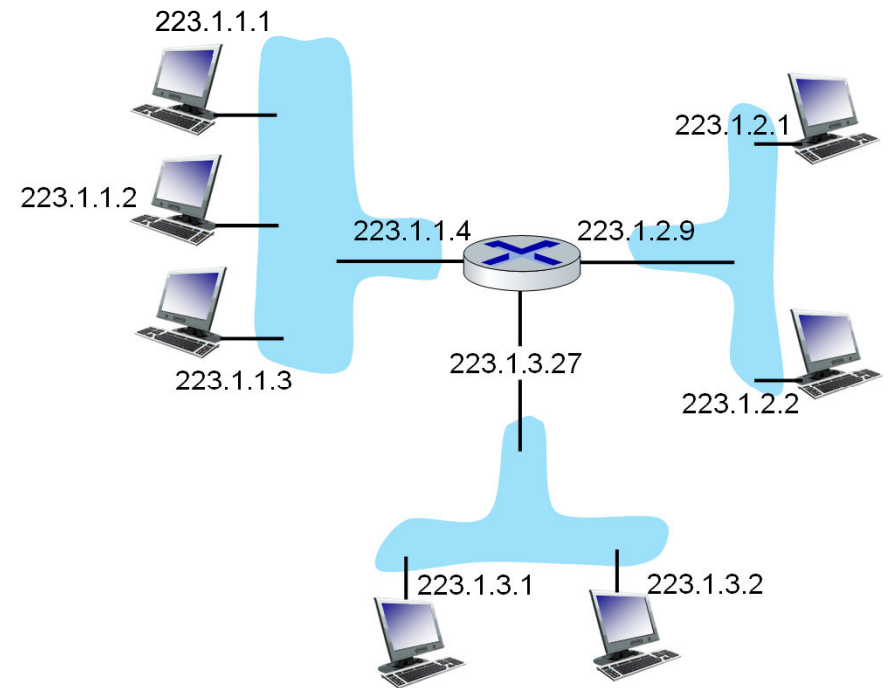
点分十进制表示法:

$$223.1.1.1 = \underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$$

Network Layer: 4-47

IP 地址: 导论

- IP 地址: 32位标示, 对主机或者路由器的接口编址
- 接口: 主机/路由器与物理链路的连接处
 - 路由器通常拥有多个接口
 - 主机典型的有1个或者2个接口 (例如: 有线以太网, 无线802.11) 也有可能拥有多个接口



点分十进制表示法:

$$223.1.1.1 = \underbrace{11011111}_{223} \underbrace{00000001}_{1} \underbrace{00000001}_{1} \underbrace{00000001}_{1}$$

Network Layer: 4-48

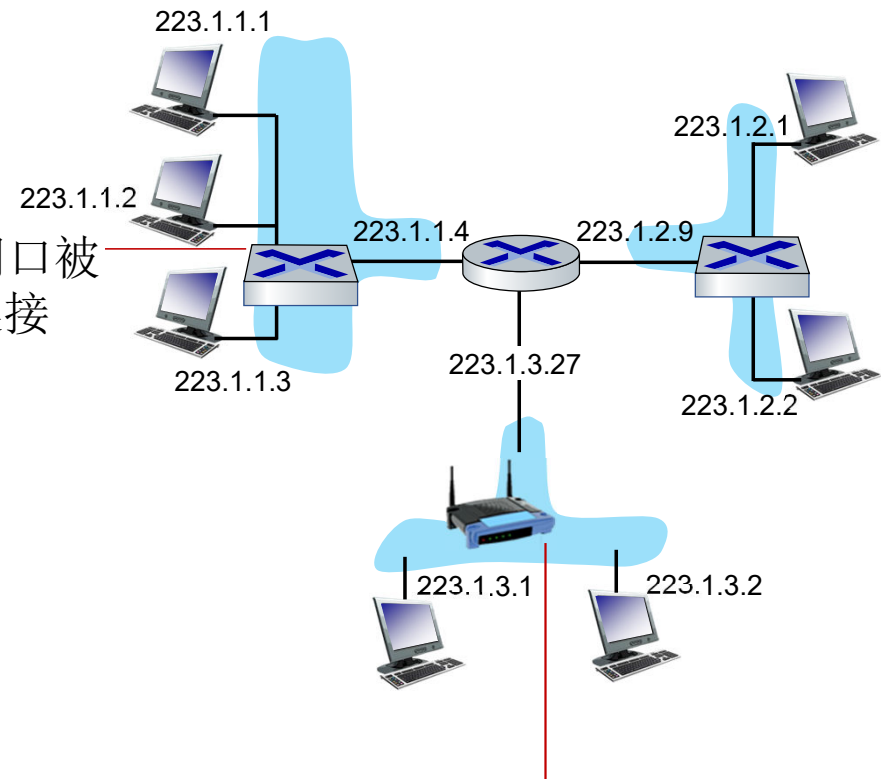
IP 地址:导论

Q: 接口之间实际是如何连接的?

A: 将会在第6, 7章中学习

目前: 无需担心一个接口是如何连接到另外一个接口 (中间没有路由器), 从而发送分组的

A: 有线的以太网口被以太网交换机连接



A: 无线WiFi接口被WiFi基站连接

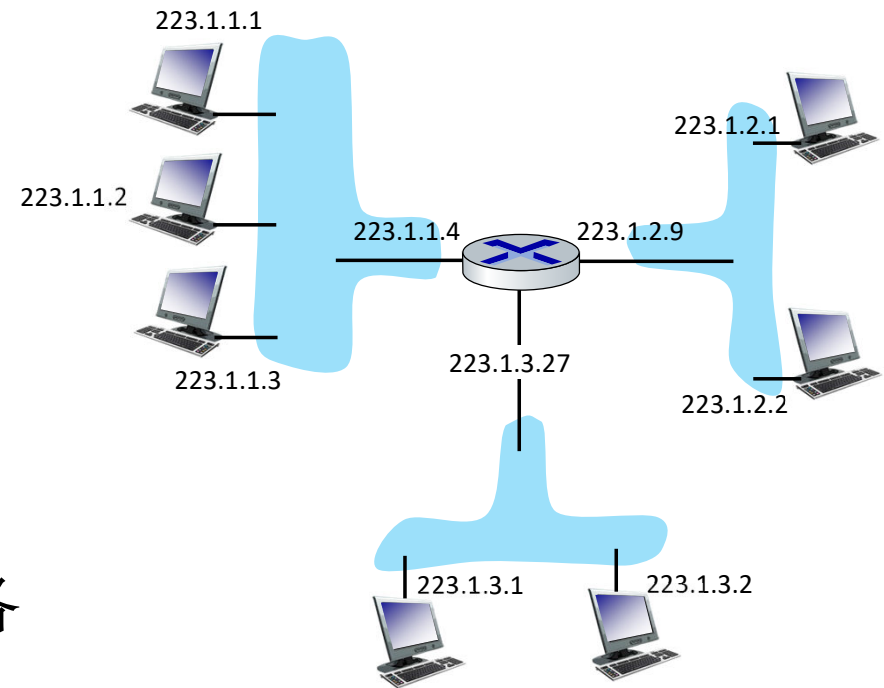
子网

■ 什么是子网?

- 无需路由器介入，子网内各主机可以在物理上相互直接到达
- 具有相同的地址前缀

■ IP地址是有结构的:

- 子网部分：在一个子网内部的各设备其地址高位部分相同
- 主机部分：余下的低位bit

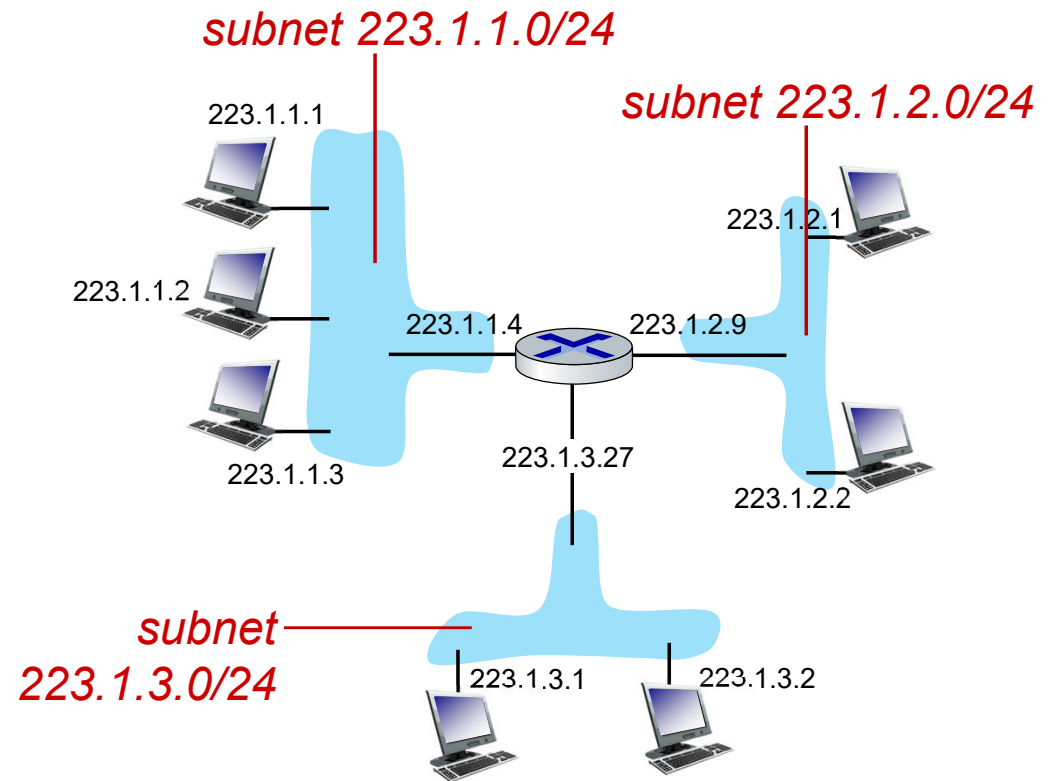


包含了3个子网的网络

子网

子网的定义方法:

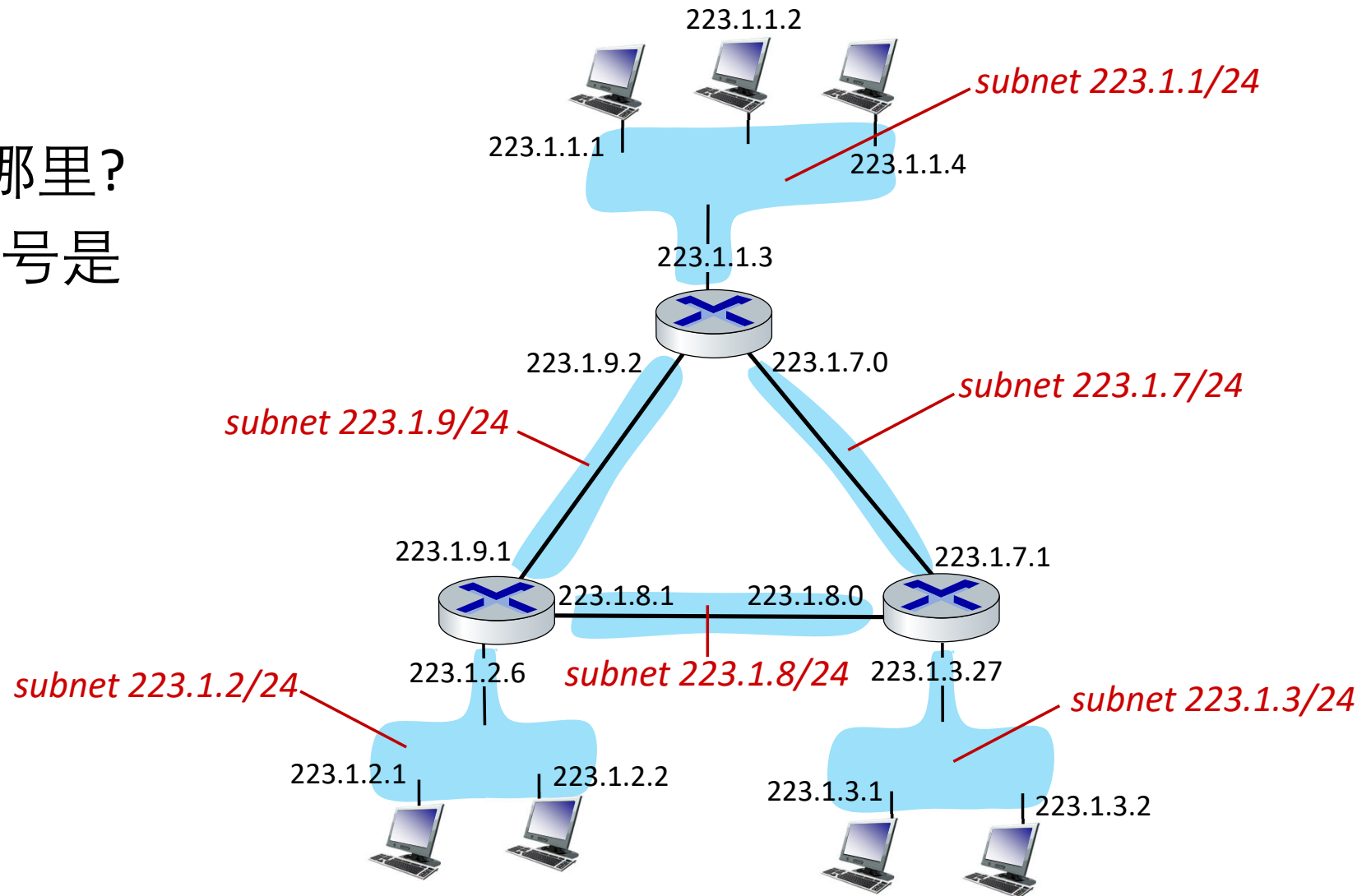
- 判断一个子网范围: 将每一个接口从主机或者路由器上分开, 构成了一个个网络的孤岛
- 每个相互独立的网络部分被叫做 **子网**



子网掩码: /24
(最高24位: IP地址中的子网部分)

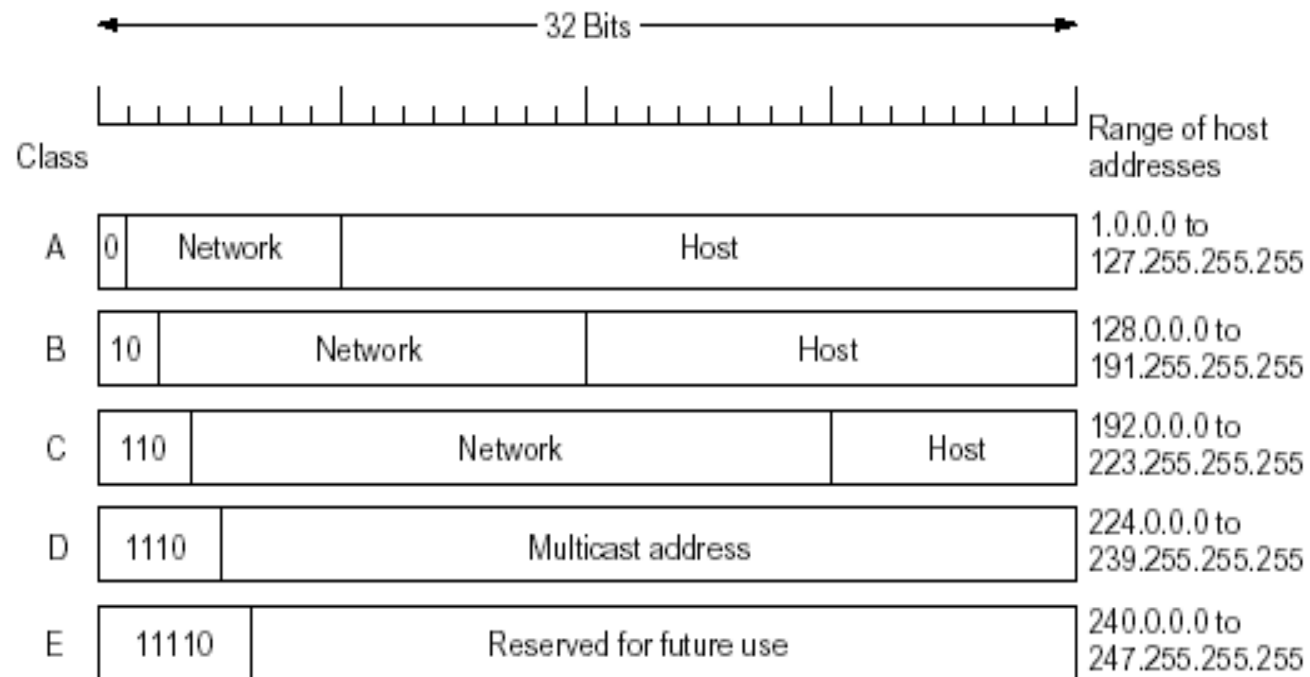
子网

- 子网在哪里?
- /24子网号是什么?



初始设计的IPv4地址分类

- Class A: 126 网络，16 M主机
- Class B: 16382网络，64 K主机
- Class C: 2M网络，254主机
- Class D: 多播
- Class E: 预留



一些特殊的IPv4地址

■ 一些约定：

- 子网部分全为0：本网络
- 主机部分全为0：本主机
- 主机部分全为1：广播地址，这个网络的所有主机

0 0																																This host								
0 0								...								0 0								Host																A host on this network
1 1																																Broadcast on the local network								
Network																1 1 1 1				...								1 1 1 1				Broadcast on a distant network								
127								(Anything)																								Loopback								

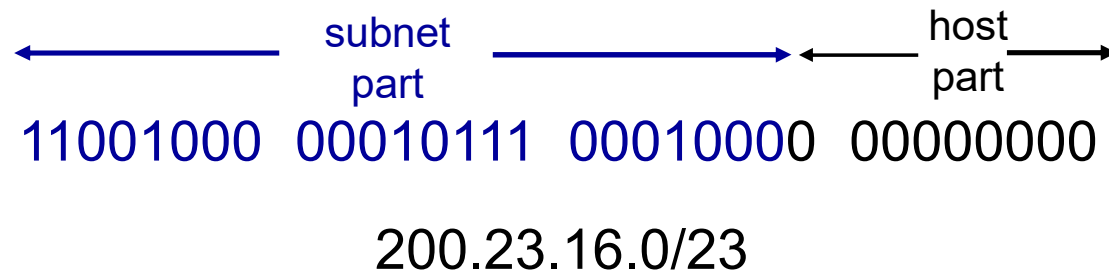
内网（专用）地址

- 专用地址：地址空间的一部份供专用地址使用
 - 永远不会被当做公用地址来分配，不会与公用地址重复
 - 只在局部网络中有意义，区分不同的设备
 - 路由器不对目标地址是专用地址的分组进行直接转发
- Class A
10. 0. 0. 0–10. 255. 255. 255
MASK 255. 0. 0. 0
 - Class B
172. 16. 0. 0–172. 31. 255. 255
MASK 255. 255. 0. 0
 - Class C
192. 168. 0. 0–192. 168. 255. 255
MASK 255. 255. 255. 0

IP地址：CIDR

CIDR: Classless InterDomain Routing (读“cider”)

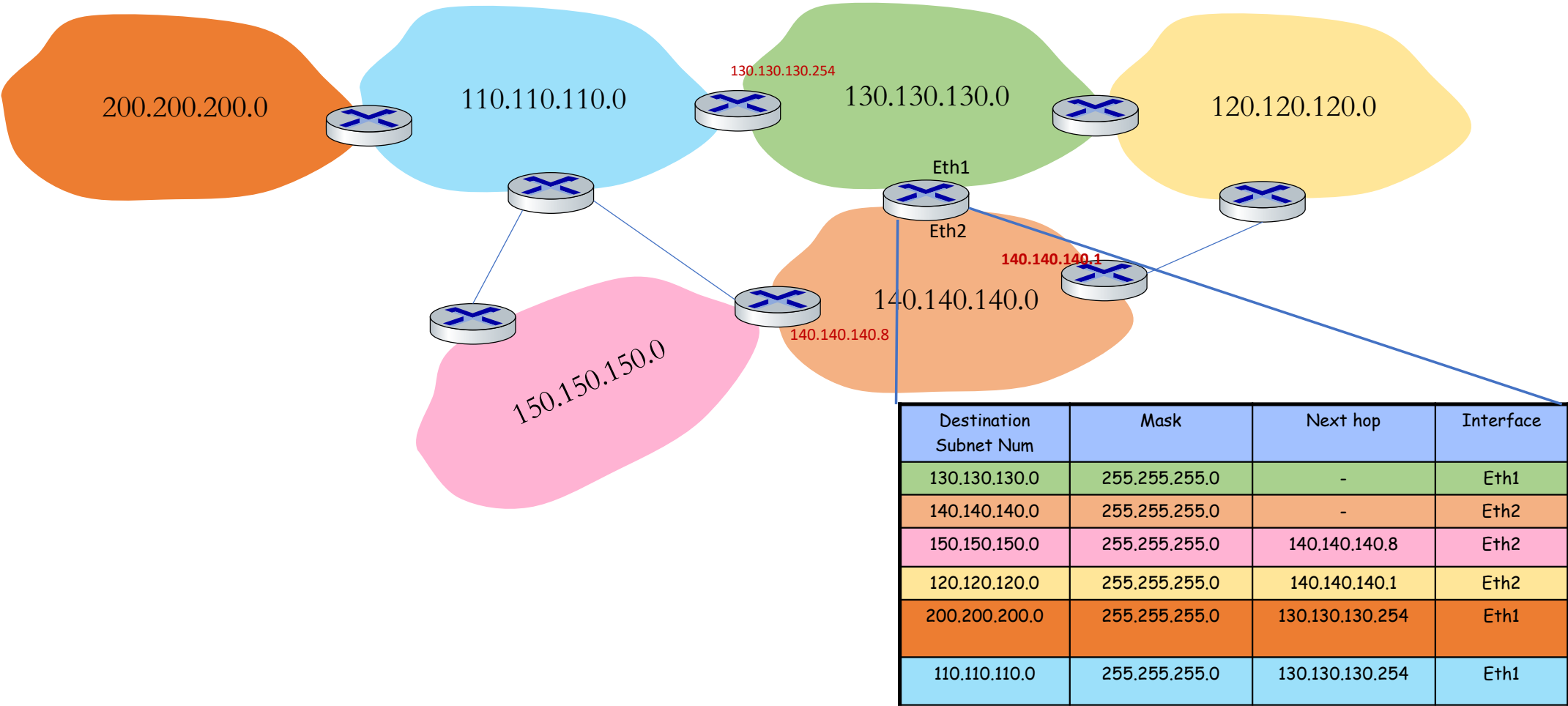
- 允许任意长度的子网部分
- 地址格式： **a.b.c.d/x**, 其中 x 是地址子网部分的bit数量



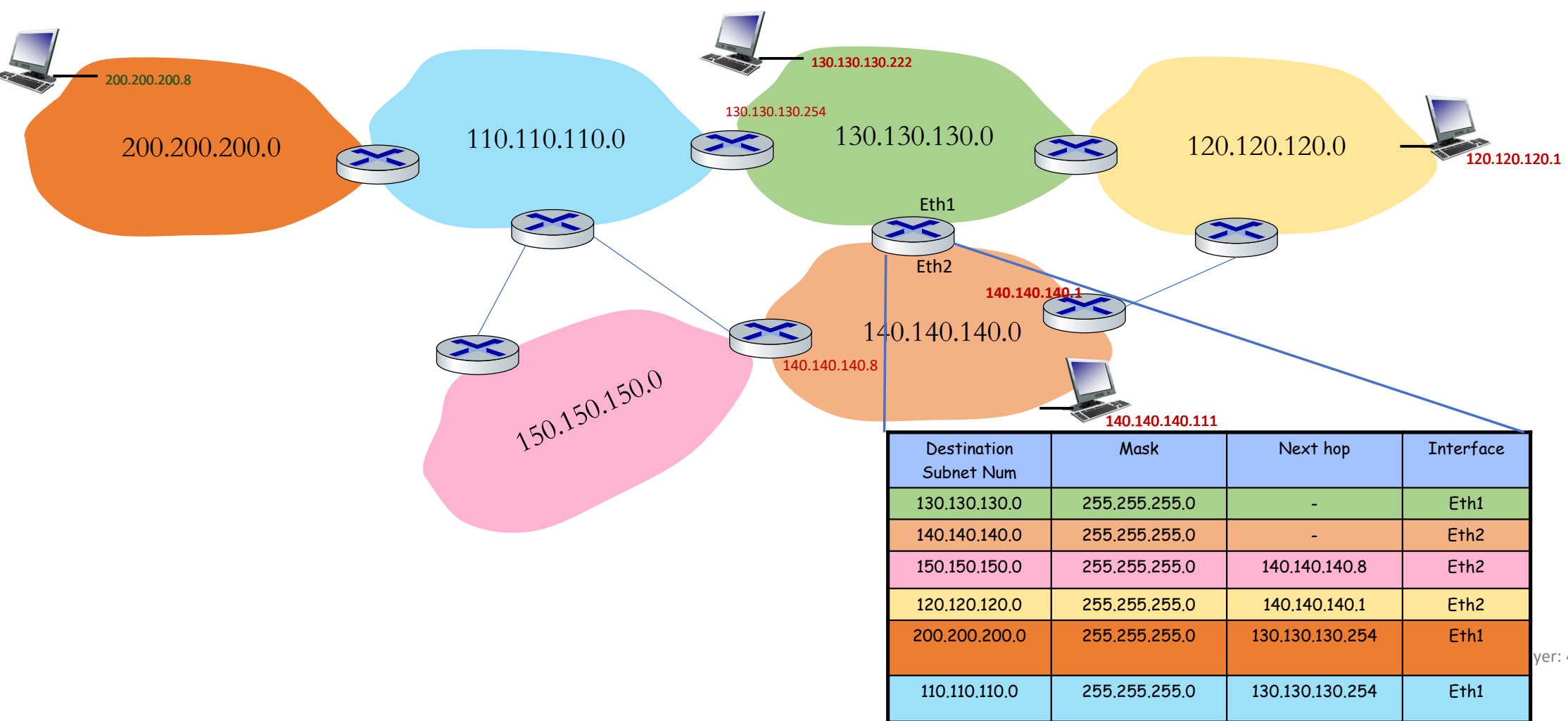
子网掩码(subnet mask)

- 32bits , 0 or 1 in each bit
 - 1: bit位置表示子网部分
 - 0: bit位置表示主机部分
- 原始的A、B、C类网络的子网掩码分别是
 - A: 255.0.0.0 : 11111111 00000000
00000000 00000000
 - B: 255.255.0.0: 11111111 11111111
00000000 00000000
 - C: 255.255.255.0: 11111111
11111111 11111111 00000000
- CIDR下的子网掩码例子:
11111111 11111111 11111100 00000000
- 另外一种表示子网掩码的表达方式
 - /#
 - 例: /22: 表示前面22个bit为子网部分

IP转发表和路由器的分组转发

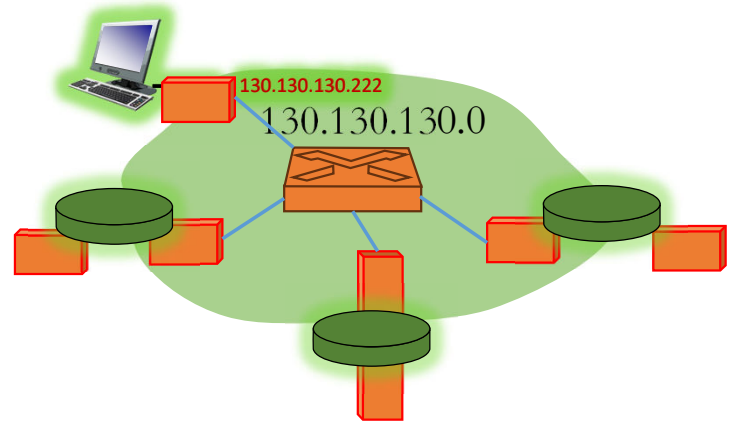
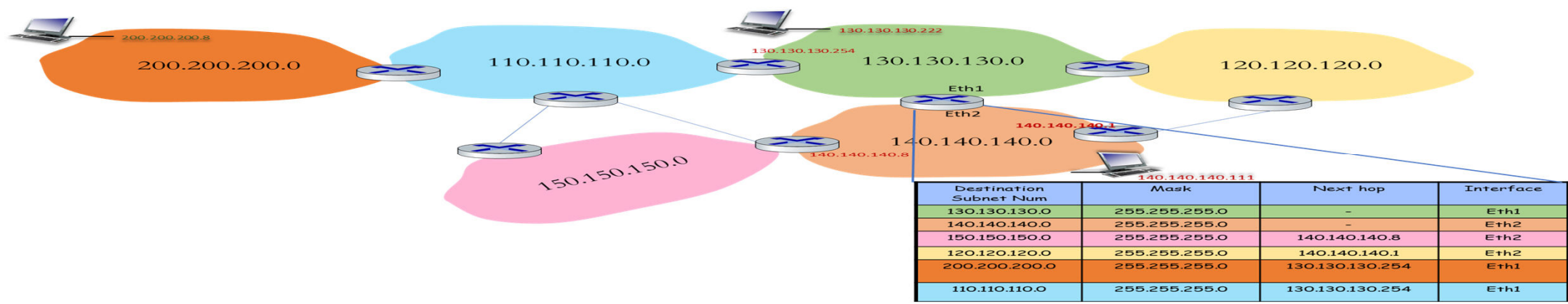


IP转发表和路由器的分组转发



IP转发表和路由器的分组转发

2025中科大自动化系



转发表（路由表）和转发算法

<i>Destination Subnet Num</i>	<i>Mask</i>	<i>Next hop</i>	<i>Interface</i>
202.38.73.0	255.255.255.192	IP_x	Lan1
202.38.64.0	255.255.255.192	IP_y	Lan2
.....			
Default	-	IP_z	Lan0

- 获得IP数据报的目标地址
- 对于转发表中的每一个表项
 - ✓ 如 $(IP\ Des\ addr) \& (mask) == destination$, 则按照表项对应的接口转发该数据报
 - ✓ 如果都没有找到,则使用默认表项转发数据报

IP 地址: 如何获取有效IP地址?

实际上有2个问题:

1. Q: 一台主机如何在一个网络内部获得一个地址（地址的主机部分给出一个唯一的主机号，网络号部分采用子网号）？
2. Q: 一个网络如何为自己获得IP地址（地址的网络部分）？

主机如何获得IP地址？

- 在配置文件中写入（例如: UNIX中/etc/rc.config）
- **DHCP: Dynamic Host Configuration Protocol**: 从服务器那里动态获得地址
 - “即插即用plug-and-play”

DHCP: Dynamic Host Configuration Protocol

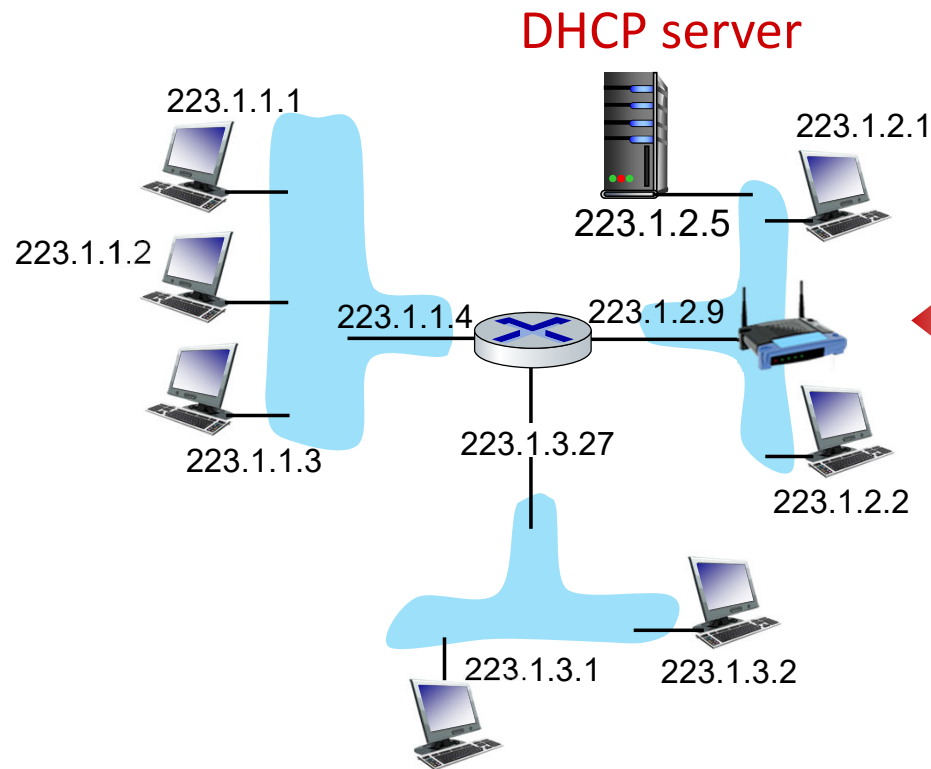
目标: 允许主机在加入网络的时候，动态地从服务器那里获得IP地址：

- ❑可以更新对主机在用IP地址的租用期-租期快到了
- ❑重新启动时，允许重新使用以前用过的IP地址
- ❑支持移动用户加入/退出该网络（短期在网）

DHCP工作概况：

- 主机广播“**DHCP discover**” 报文[可选]
- DHCP 服务器用“**DHCP offer**” 提供报文响应[可选]
- 主机请求IP地址：发送“**DHCP request**” 报文
- DHCP服务器发送地址：“**DHCP ack**” 报文

DHCP client-server 场景



一般地DHCP服务器和路由器捆绑在一起，服务于路由器连接的所有子网的地址分配



进入的**DHCP客户端**需要该子网的一个IP地址

DHCP client-server 场景

DHCP server: 223.1.2.5



DHCP discover

Broadcast: is there a
DHCP server out there?

Arriving client



DHCP offer

Broadcast: I'm a DHCP
server! Here's an IP
address you can use

DHCP request

Broadcast: OK. I would
like to use this IP address!

DHCP ACK

Broadcast: OK. You've
got that IP address!

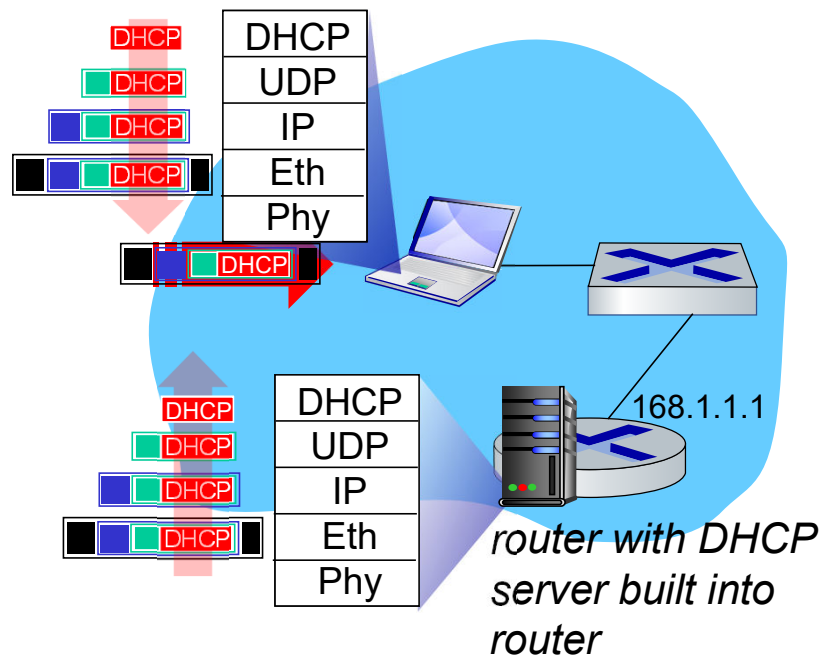
这2个步骤可以被跳过，
“如果一个客户端记住之
前被分配的地址，并且愿
意接着使用该地址”
[RFC 2131]

DHCP: 不仅仅可以分配IP地址

DHCP不仅仅可以返回在一个子网中分配的IP地址：

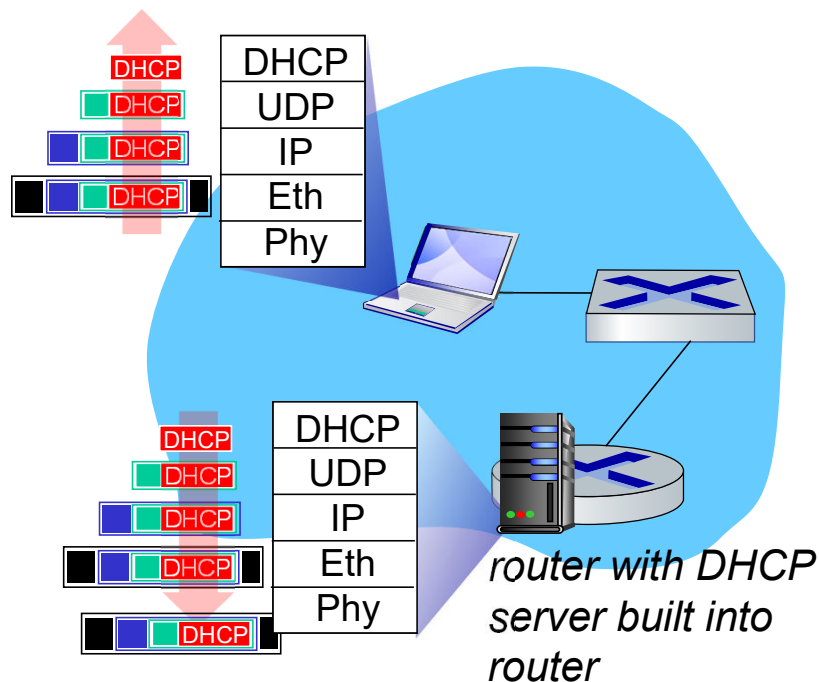
- **默认网关**：第一跳路由器IP地址（对于客户端来说要将分组发向外部网络所需要经过的第一跳路由器的IP地址）
- **DNS服务器**的域名和IP地址
- **子网掩码**（指示IP地址中哪些位是网络号，哪些位是主机号）

DHCP: 例子



- 联网笔记本需要获取自己的IP地址、第一跳路由器地址和DNS服务器：采用DHCP协议
- ❖ DHCP 请求被封装在UDP段中，封装在IP数据报中，封装在以太网的帧中
- ❖ 以太网帧在局域网范围内广播（dest: FFFFFFFFFFFFFFFF），被运行DHCP服务的路由器收到
- ❖ DHCP服务器处：以太网帧解封装成IP，IP解封装成UDP，解封装成DHCP

DHCP: 例子



- DHCP服务器生成DHCP ACK，包含客户端的IP地址、第一跳路由器的IP地址和DNS域名服务器的IP地址
- ❖ DHCP服务器封装的报文所在的帧转发到客户端，在客户端解封装成DHCP报文
- ❖ 客户端知道它自己的IP地址、DNS服务器域名和IP地址、第一跳路由器的IP地址

IP 地址： 如何获得一块网络地址？

Q: 如何获得一个网络的子网部分？

A: 从ISP已获得的地址块中分配一个小地址块

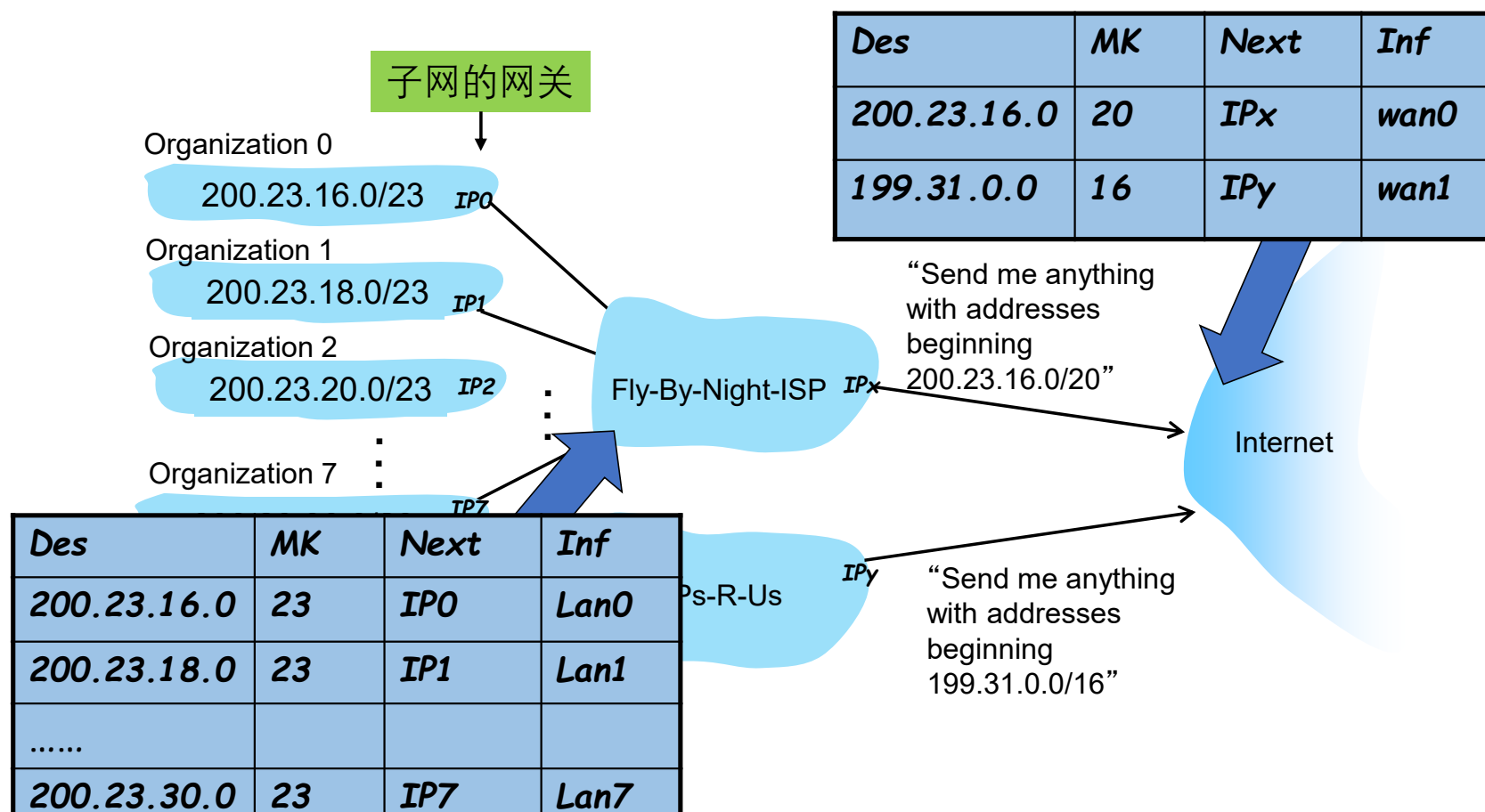
ISP's block 11001000 00010111 00010000 00000000 200.23.16.0/20

ISP可以将其拥有的地址分为8个地址块：

Organization 0	<u>11001000 00010111 00010000</u>	00000000	200.23.16.0/23
Organization 1	<u>11001000 00010111 00010010</u>	00000000	200.23.18.0/23
Organization 2	<u>11001000 00010111 00010100</u>	00000000	200.23.20.0/23
...			
Organization 7	<u>11001000 00010111 00011110</u>	00000000	200.23.30.0/23

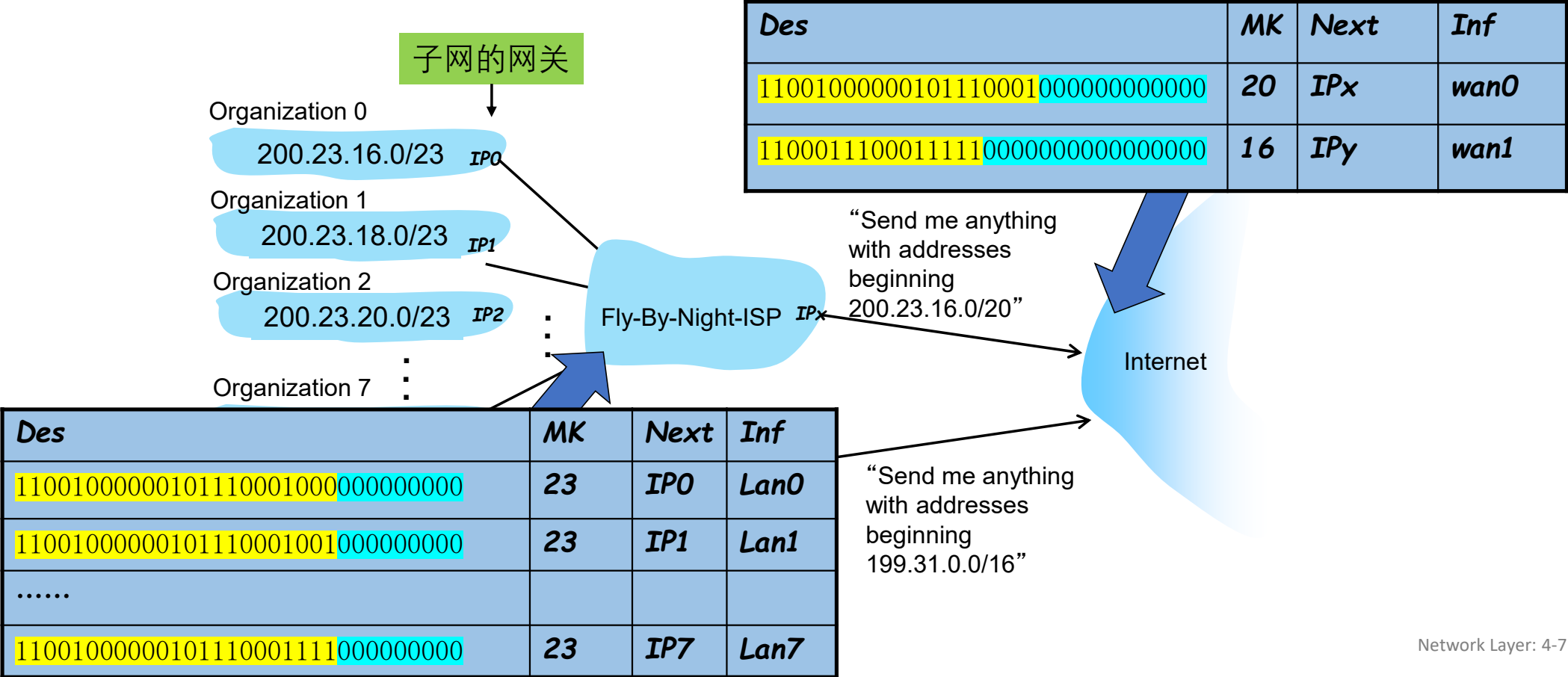
层次编址：路由聚集

层次编址允许路由信息的有效散播：



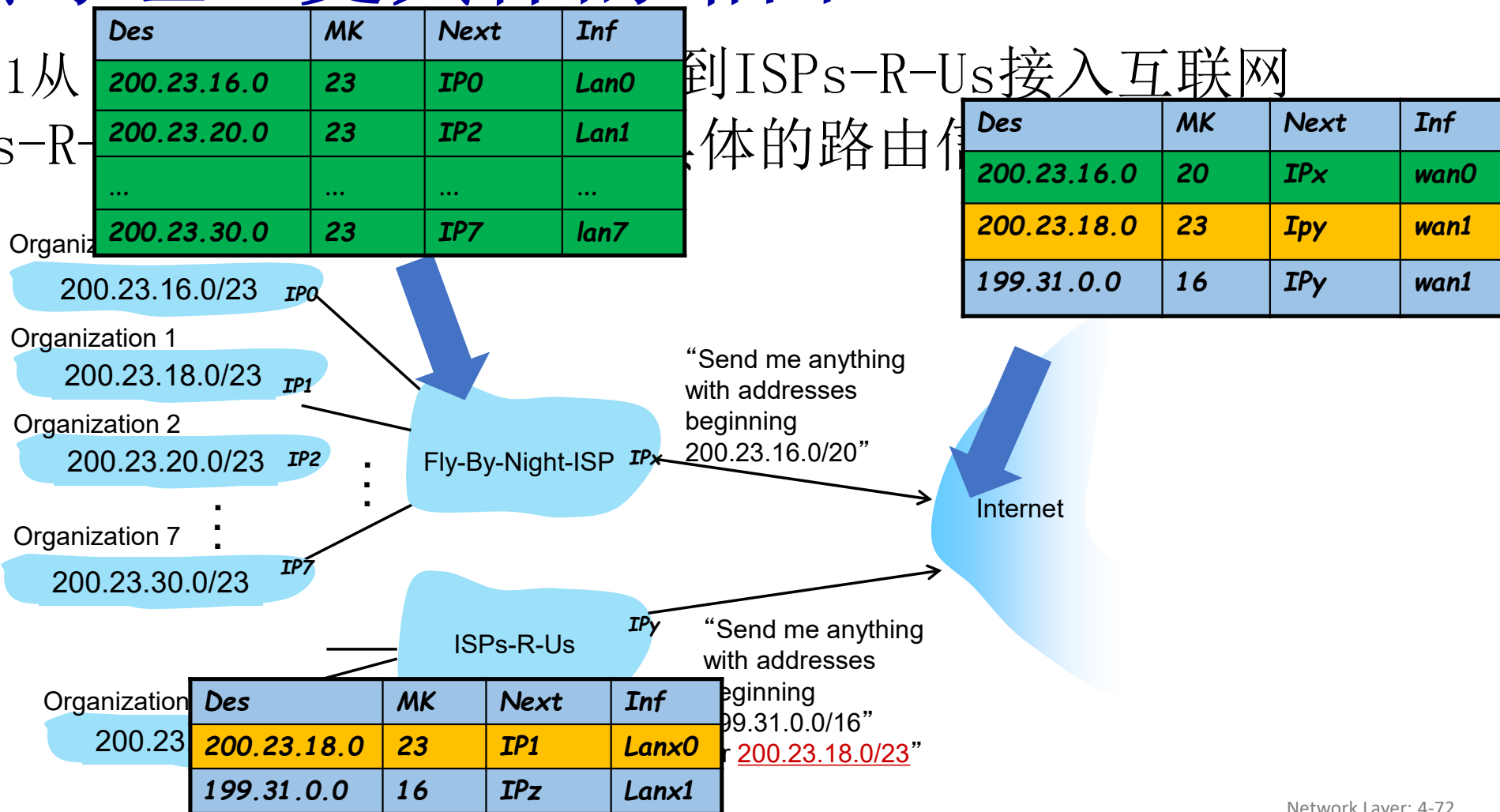
层次编址：路由聚集

层次编址允许路由信息的有效广播：



分层寻址：更具体的路由

- 机构1从
- ISPs-R-Us接入互联网



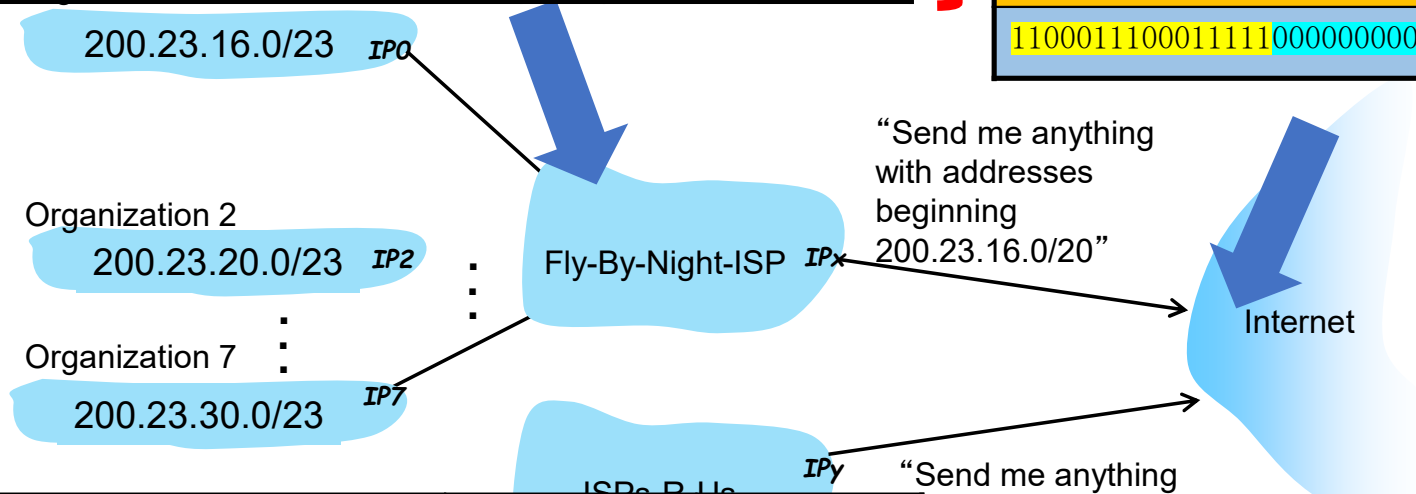
分层寻址：更具体的路由

- 主机
- ISP

Des	MK	Next	Inf
11001000000010111000100000000000	23	IP0	Lan0
11001000000010111000101000000000	23	IP2	Lan1
...	...	...	...
11001000000010111000111100000000	23	IP7	lan7

ISPs-R-Us接入互联网

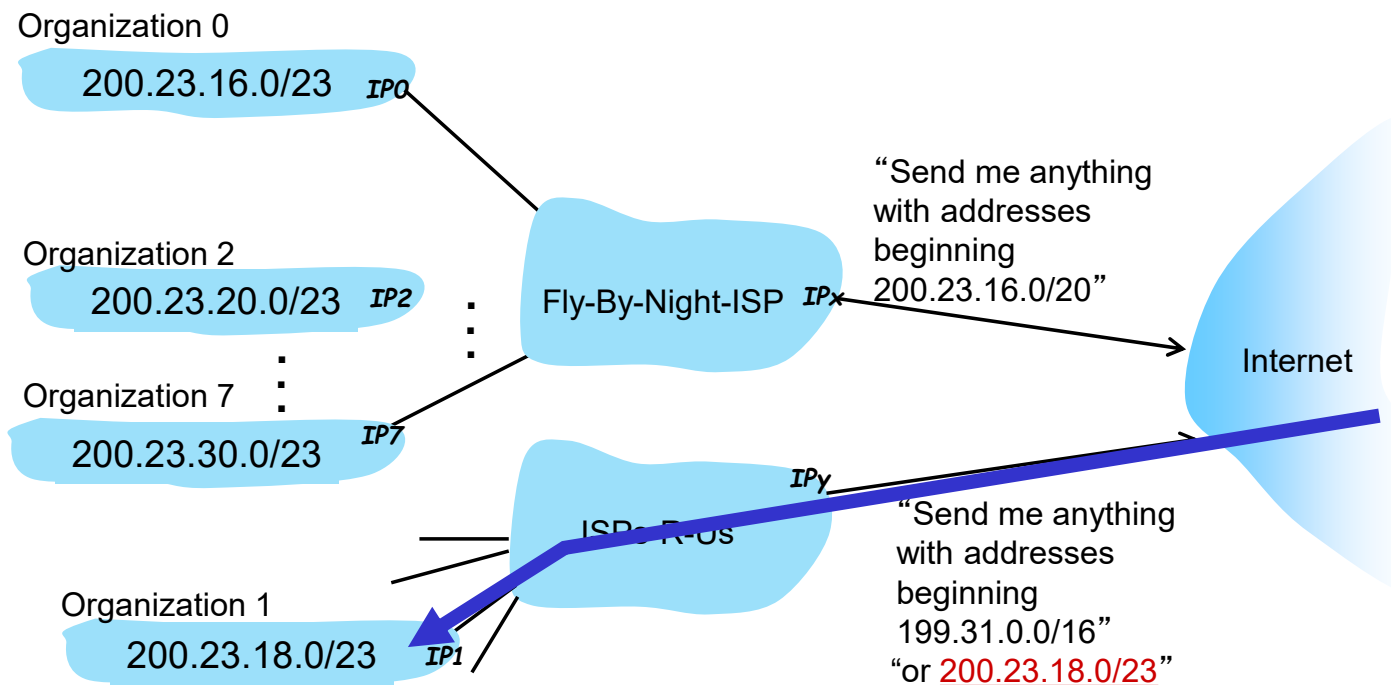
Des	MK	Next	Inf
11001000000010111000100000000000	20	IPx	wan0
11001000000010111000100100000000	23	Ipy	wan1
11000111000111110000000000000000	16	IPy	wan1



Des	MK	Next	Inf
11001000000010111000100100000000	23	IPx	Lanx0
11000111000111110000000000000000	16	IPy	Lanx1

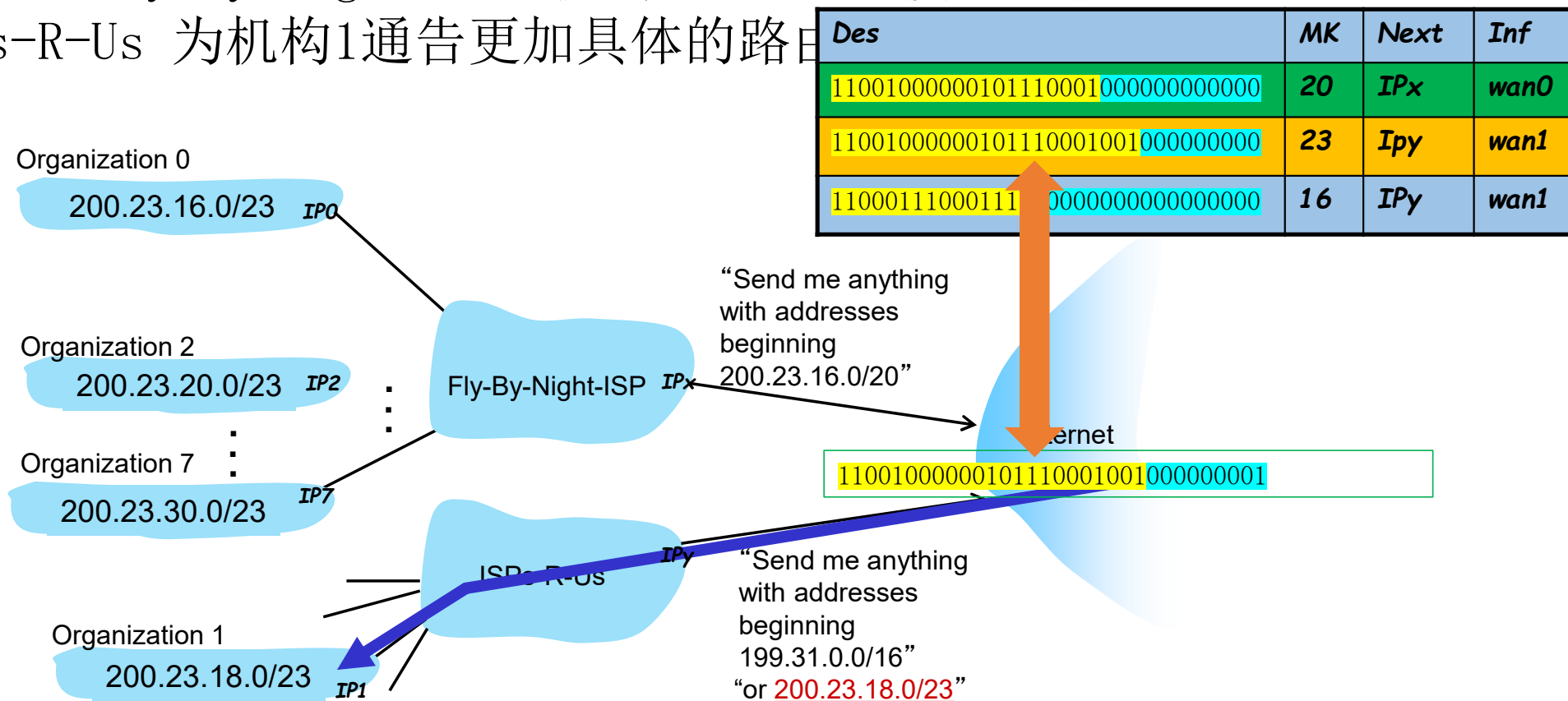
分层寻址：更具体的路由

- 机构1从 Fly-By-Night-ISP 移到ISPs-R-Us接入互联网
- ISPs-R-Us 为机构1通告更加具体的路由信息



分层寻址：更具体的路由

- 机构1从 Fly-By-Night-ISP 移到ISPs-R-Us接入互联网
- ISPs-R-Us 为机构1通告更加具体的路由



IP 地址: 最后的话...

Q: 一个ISP如何获得一个地址块?

A: ICANN: Internet Corporation for Assigned Names and Numbers
<http://www.icann.org/>

- 分配IP地址，通过5个地区注册中心RR（他们接着再分配给本地注册中心）
- 管理DNS根区域，包括单个TLD（.com、.edu...）管理的委派

Q: 有足够的32位IPv4地址吗?

- ICANN在2011年为RR分配完最后一块IPv4地址
- NAT（后续）有助于IPv4地址空间的耗尽
- IPv6拥有128位地址空间

“谁知道我们需要多少地址空间？”Vint Cerf（反思将IPv4地址设为32位长的决定）

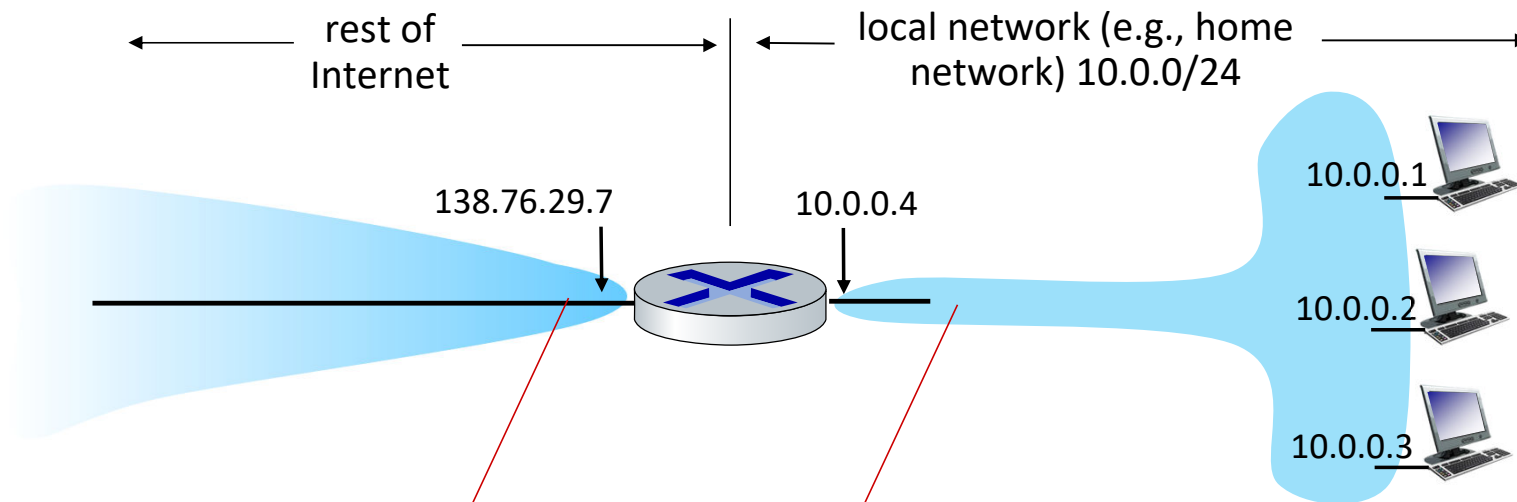
网络层：“数据平面” 提纲

- 网络层：概述
 - 数据平面
 - 控制平面
- 路由器内部结构
 - 输入端口，交换机构，输出端口
 - 缓存管理，调度
- IP: the Internet Protocol
 - 数据报格式
 - 编址
 - 网络地址转换：NAT
 - IPv6
- 通用转发，SDN
 - 匹配+行动
 - OpenFlow: 匹配+行动 运行
- 中间盒



NAT: network address translation

NAT:从外界来看本地网络的所有设备只共享一个IPv4地址，



所有的本地网络外出数据报具有同样的源IP地址：138.76.29.7，但是它们的源端口号不同

来自该网络或者去往该网络的数据报都把地址：10.0.0/24作为其源和目标地址（与正常情况一样）

NAT: network address translation

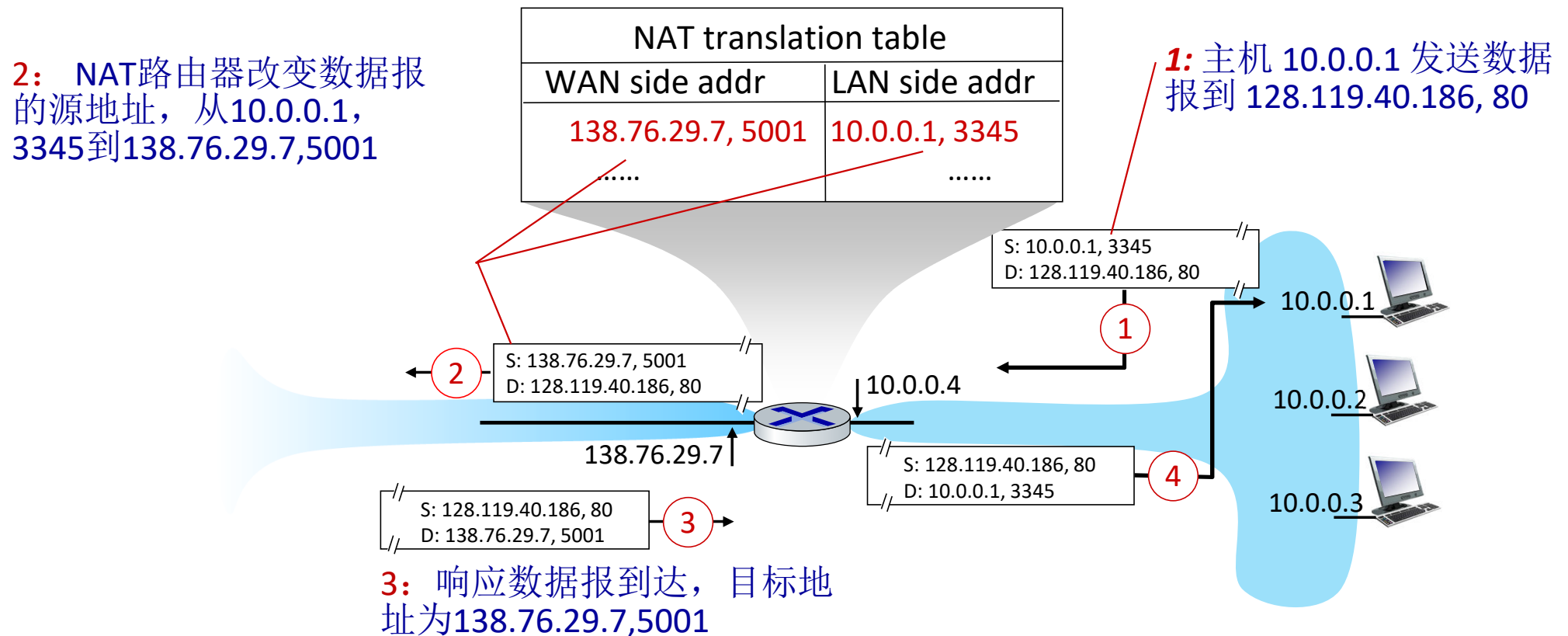
- 所有本地网络的设备都拥有32位的，在”私有”地址范围的IP地址(前缀为10/8, 172.16/12, 192.168/16)，该私有地址仅仅被用于本地网络
- 好处：
 - 仅需要从ISP处为所有设备获得1个有效IP地址
 - 改变本地网络设备的地址，无须通知外界
 - 改变ISP（外网地址变化）而不需要改变内部的设备地址
 - 安全：本地网络内部的设备不能够外界直接被寻址到，使用的本地地址对外界不可见

NAT: network address translation

实现: NAT 路由器必须 (透明地):

- 外出的数据报: 替换每个外出数据报的 (源IP地址, 源端口号), 将之变换成 (NAT外部IP地址, 新端口)
 - 远程的客户端/服务器将会回应该数据报, 目标地址: (NAT外网IP地址, 新端口号)
- 记下 (在NAT转换表中) 每一个 (源IP内网地址, 源端口号) 到 (NAT 外网IP地址, 新端口号) 的映射对
- 进来的数据报: 替换每个到达该网络数据报的目标字段 (NAT外网IP地址, 新端口号), 将之替换成在NAT 表格中的 (源内网IP地址, 源端口号)

NAT: network address translation



NAT: network address translation

- NAT一直备受争议:
 - 路由器“应该”只处理至多3层的信息（port为4层信息）
 - 地址“短缺”应该被IPv6来解决
 - 违反了“端到端”原则（复杂性放到网络边缘主机上，而NAT的端口号被网络层设备操纵修改）
 - NAT穿越：如果客户端需要主动连接在NAT路由器之后的服务器，该如何做？
- 但是NAT已经是一个事实了：
 - 广泛存在于家庭网络、机构网络，4G/5G蜂窝网络

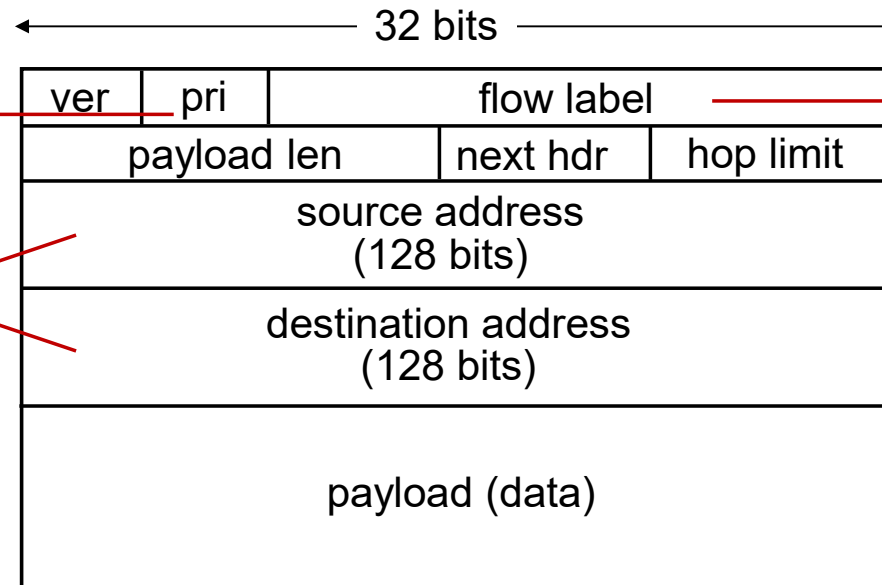
IPv6： 动机

- **初始动机：** 32-bit地址空间将会被完全分配完毕
- 另外的动机：
 - 采用固定长度的头部可以加速IP数据报的处理和转发
 - 支持对“流”进行的不同网络层处理

IPv6 数据报格式

优先级: 在流的数据报间标识其优先级

128-bit
IPv6 地址



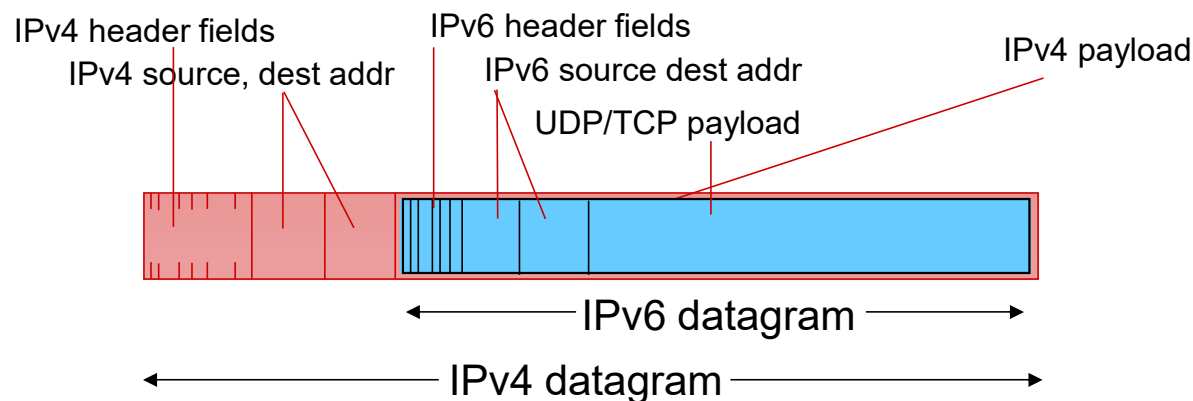
流标签: 标识在同一个“流”中的数据报 (“流”的概念没有被很好的定义).

哪些被遗弃了 (与IPv4相比较):

- 没有校验和 (在路由器上加速对数据报的处理)
- 没有分段/重组
- 没有选项 (在路由器上通过上层协议, 下一个头部 字段获得)

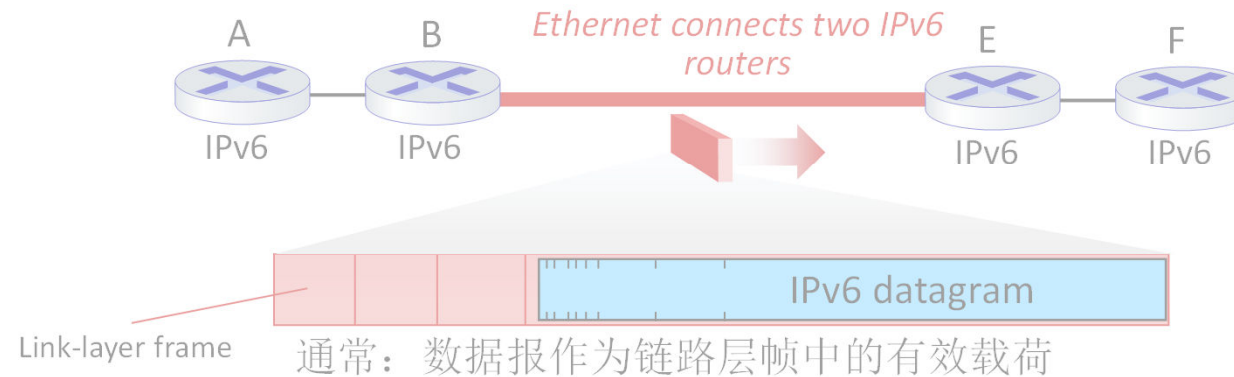
IPv4到IPv6的过渡

- 所有的路由器都同时升级不现实
 - 没有“flag days”
 - 混合了IPv4和IPv6路由器的网络如何运行？
- **隧道**: IPv6 数据报作为IPv4的载荷字段封装在IPv4数据报中，在IPv4 的路由器之间传输 (“分组被封装在分组中”)
 - 隧道技术在其他环境中广泛使用（4G/5G）

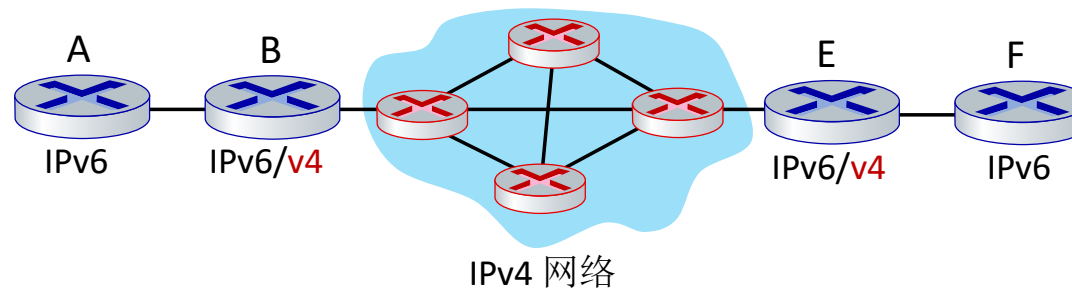


隧道和封装

以太网连接2个
IPv6路由器:

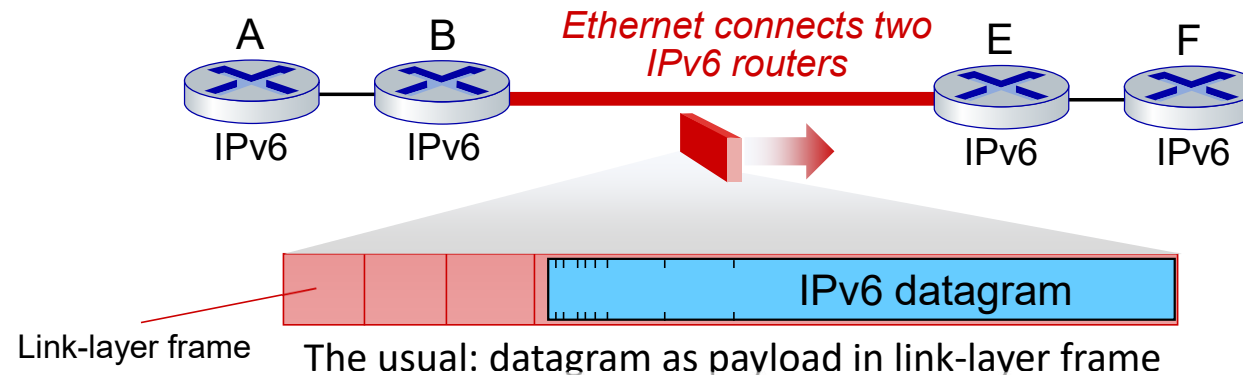


IPv4网络连接2个
IPv6路由器

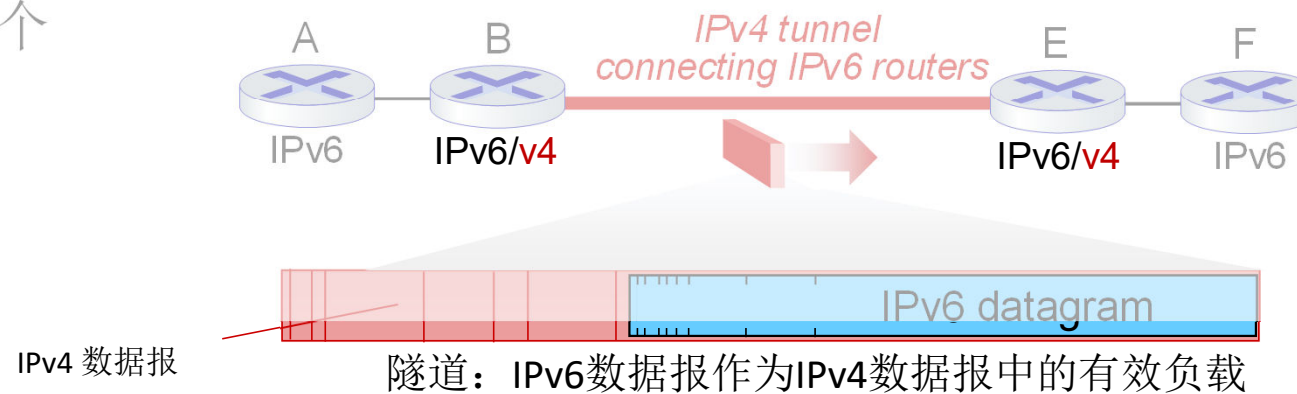


隧道和封装

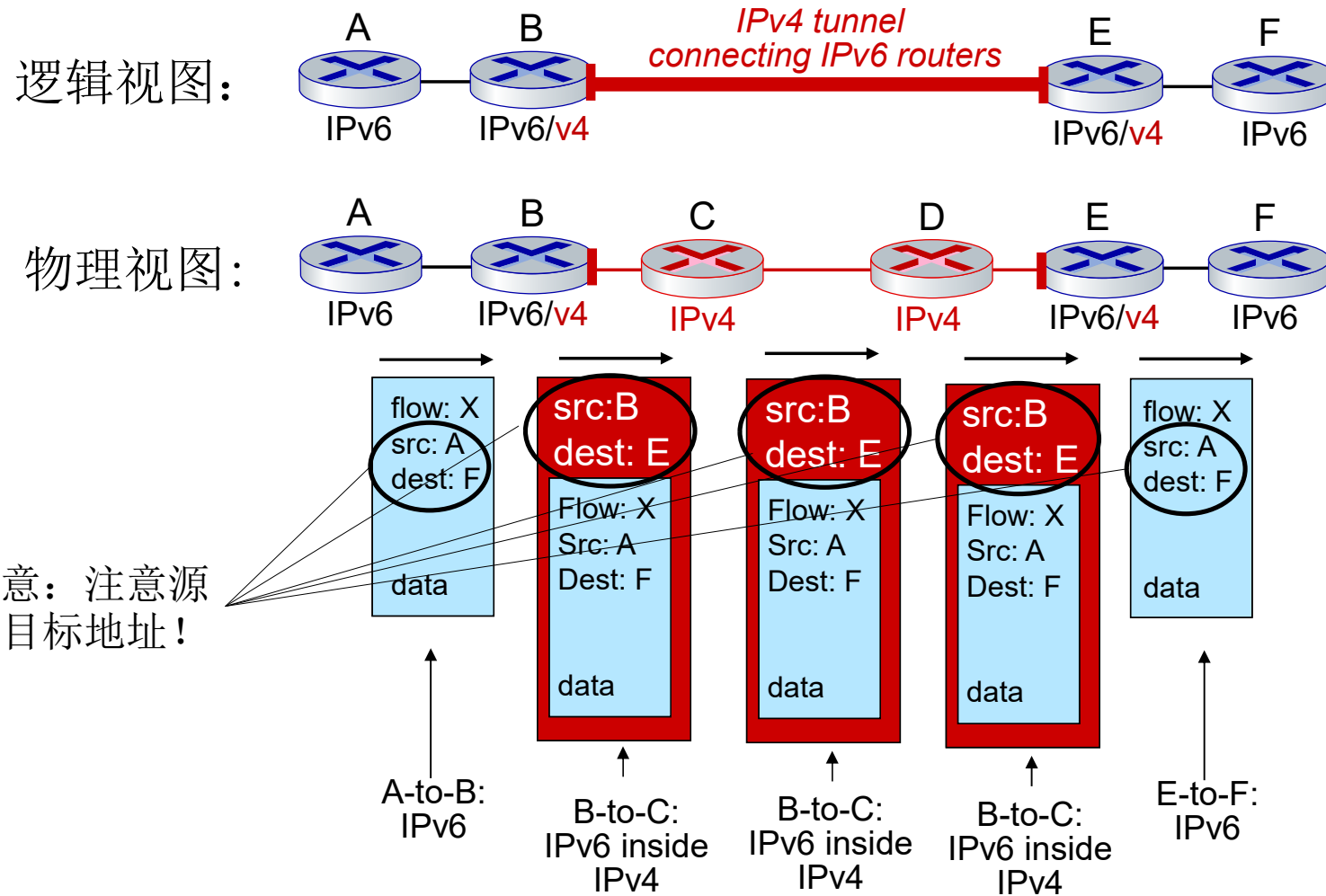
以太网连接2个
IPv6路由器:



IPv4网络连接2个
IPv6路由器



隧道

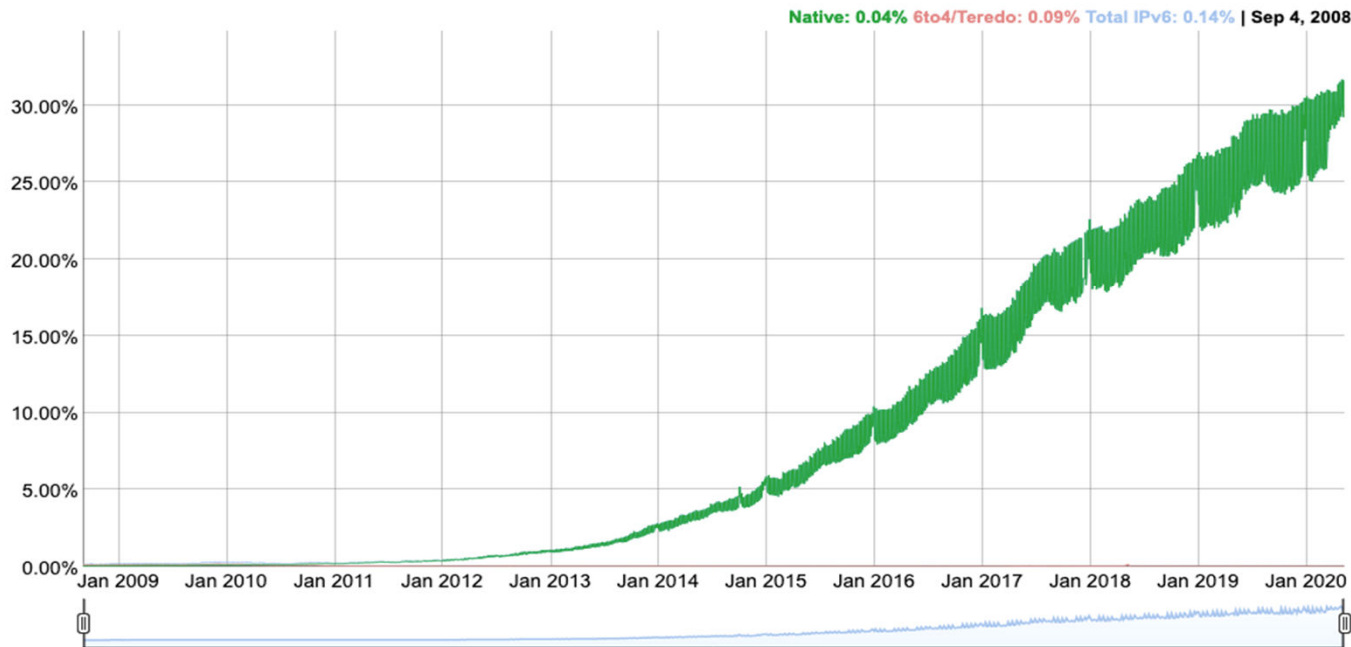


IPv6：部署情况

- Google¹: ~ 30% 客户端通过IPv6接入谷歌的服务
- NIST: 1/3的US政府域下设备是IPv6使能的

IPv6 Adoption

We are continuously measuring the availability of IPv6 connectivity among Google users. The graph shows the percentage of users that access Google over IPv6.



1

<https://www.google.com/intl/en/ipv6/statistics.html>

IPv6: 部署情况

- Google¹: ~30%的客户端通过IPv6访问服务
- NIST: 1/3的美国政府域支持IPv6
- 需要很长时间去部署, 使用
 - 25年了, 还在持续!
 - 想象一下25年内在应用层的改变: WWW, 社交媒体, 流媒体, 游戏, 远程交互
 - *Why?*

¹ <https://www.google.com/intl/en/ipv6/statistics.html>

网络层：“数据平面” 提纲

- 网络层：概述
 - 数据平面
 - 控制平面
- 路由器内部结构
 - 输入端口，交换机构，输出端口
 - 缓存管理，调度
- IP: the Internet Protocol
 - 数据报格式
 - 编址
 - 网络地址转换：NAT
 - IPv6
- 通用转发，SDN
 - 匹配+行动
 - OpenFlow: 匹配+行动的运行
- 中间盒



通用转发：匹配和行动

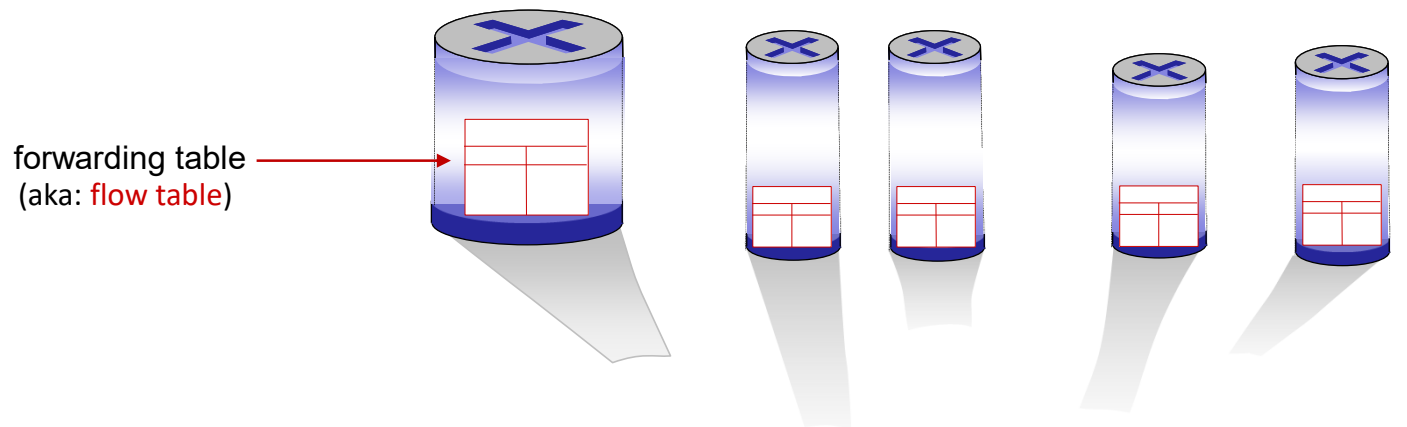
回顾：每个路由器包含有**转发表** (aka: **流表**)

- “**匹配+行动**” 抽象，匹配进来分组的若干bit, 采取行动

- **基于目标的转发**：转发基于目标IP地址

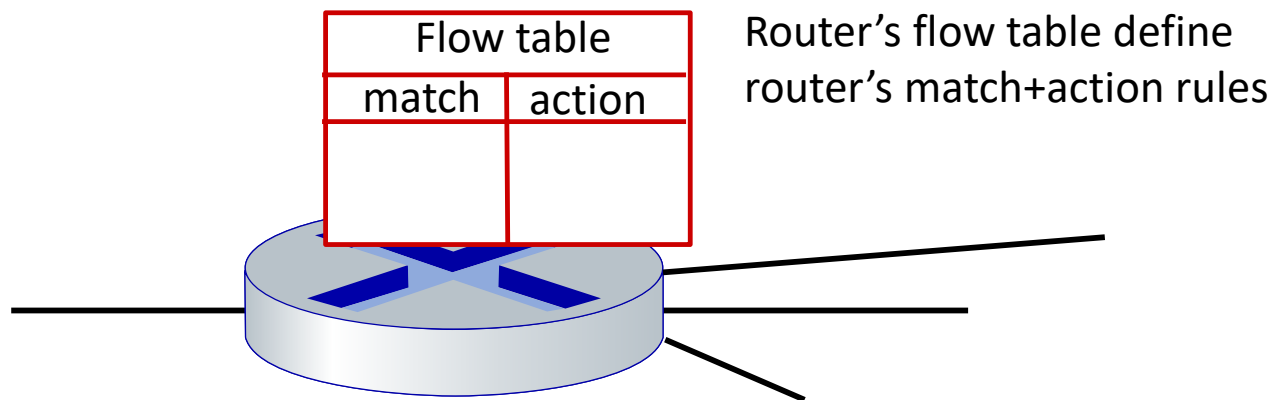
- **通用转发**：

- 很多头部字段可以用来决定行动
- 很多可能的行动： 丢弃/拷贝/修改/记录



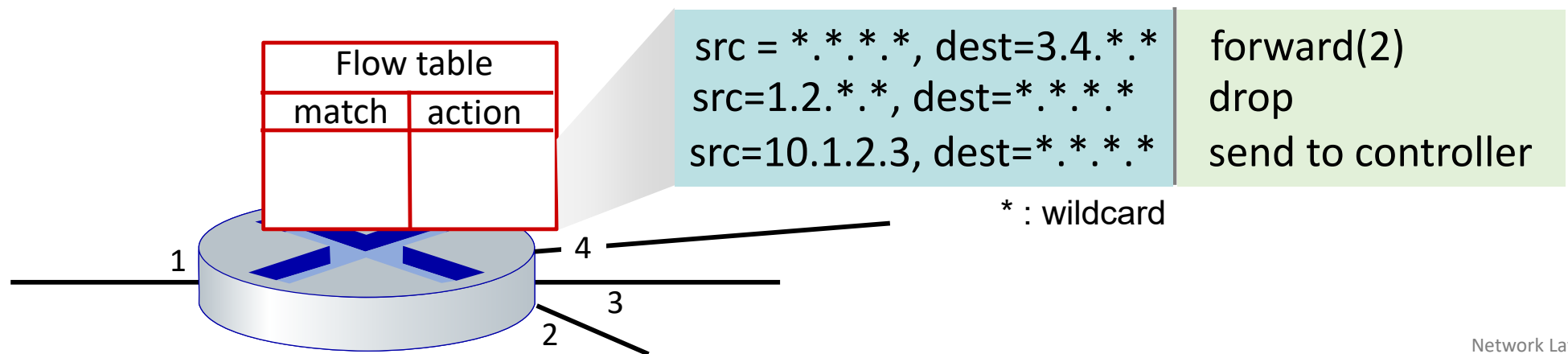
流表抽象

- **流**: 被头部字段的值来决定（在链路，网络，传输层的头部字段）
- **通用转发**: 简单的分组处理规则
 - **匹配**: 分组头部字段值的模式
 - **行动**: 对于每个匹配上的分组：丢弃，转发，修改，匹配后的分组交控制器
 - **优先级**: 消除重叠模式的歧义，多个流表表项被匹配上了，用优先级高的
 - **计数器**: #bytes and #packets

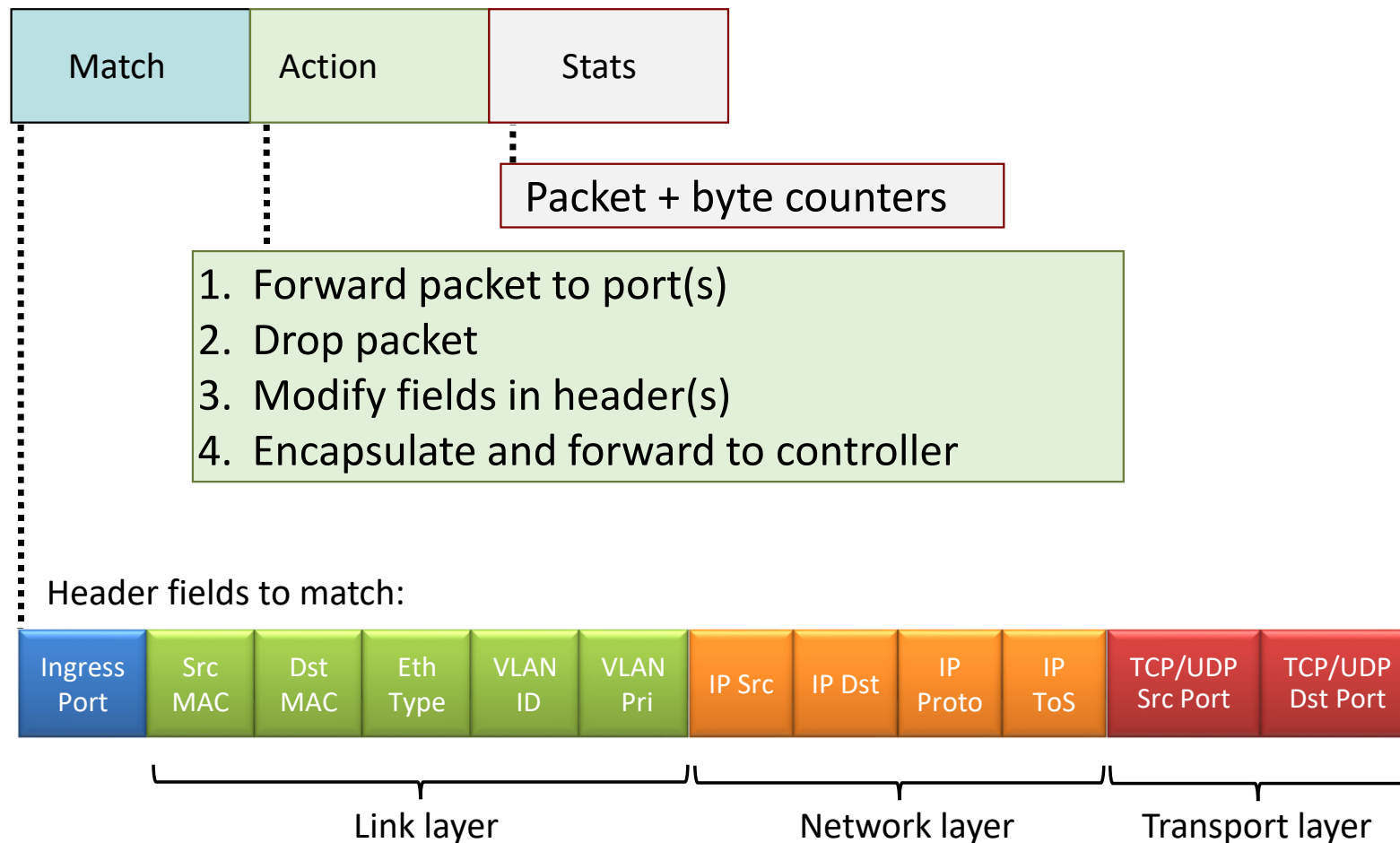


流表抽象

- **流**: 被头部字段的值来决定（在链路，网络，传输层的头部字段）
- **通用转发**: 简单的分组处理规则
 - **匹配**: 分组头部字段值的模式
 - **行动**: 对于每个匹配上的分组：丢弃，转发，修改，匹配后的分组交控制器
 - **优先级**: 消除重叠模式的歧义，多个流表表项被匹配上了，用优先级高的
 - **计数器**: #bytes and #packets



OpenFlow: 流表表项



OpenFlow: 例子

基于目标的转发:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	51.6.0.8	*	*	*	*	port6

IP数据报其目标IP地址为 51.6.0.8，应该通过路由器端口6转发

防火墙:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	*	*	*	22	*	drop

阻塞 (不转发) 所有的具备目标TCP端口为22的数据报 (ssh 端口 #)

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	128.119.1.1	*	*	*	*	*	drop

阻塞 (不转发) 所有的由128.119.1.1主机发送的分组

OpenFlow: 例子

层2基于目标的转发:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	22:A7:23:11:E1:02	*	*	*	*	*	*	*	*	*	port3

2层的具备目标MAC地址为：22：A7:23:11:E1:02的帧，应该通过端口3转发

OpenFlow 抽象

- **匹配行动**: 抽象统一了不同类型的设备

路由器

- **匹配**: 目标IP地址的最长前缀匹配
- **行动**: 通过一个端口对应的链路转发

交换机

- **match**: 目标MAC地址
- **行动**: 转发或者泛洪

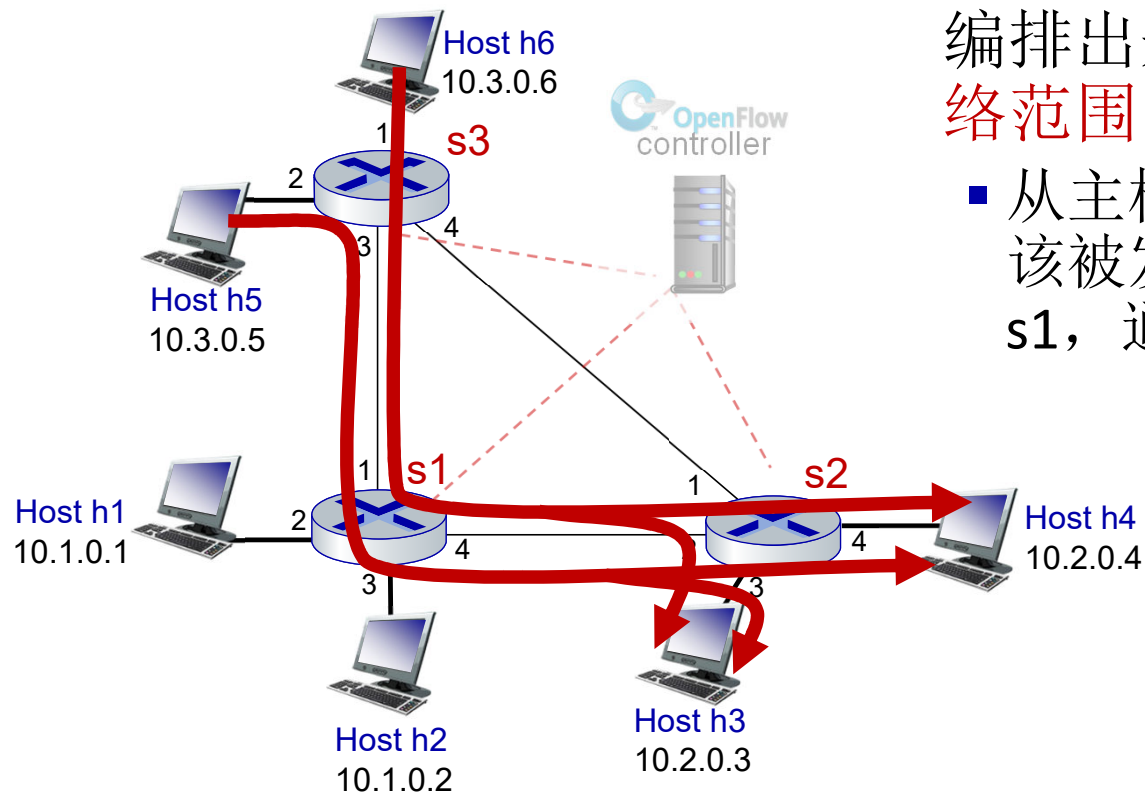
防火墙

- **匹配**: IP地址和TCP/UDP端口号
- **行动**: 允许或者拒绝

NAT

- **匹配**: IP地址和端口号
- **行动**: 写入新的IP地址和端口号

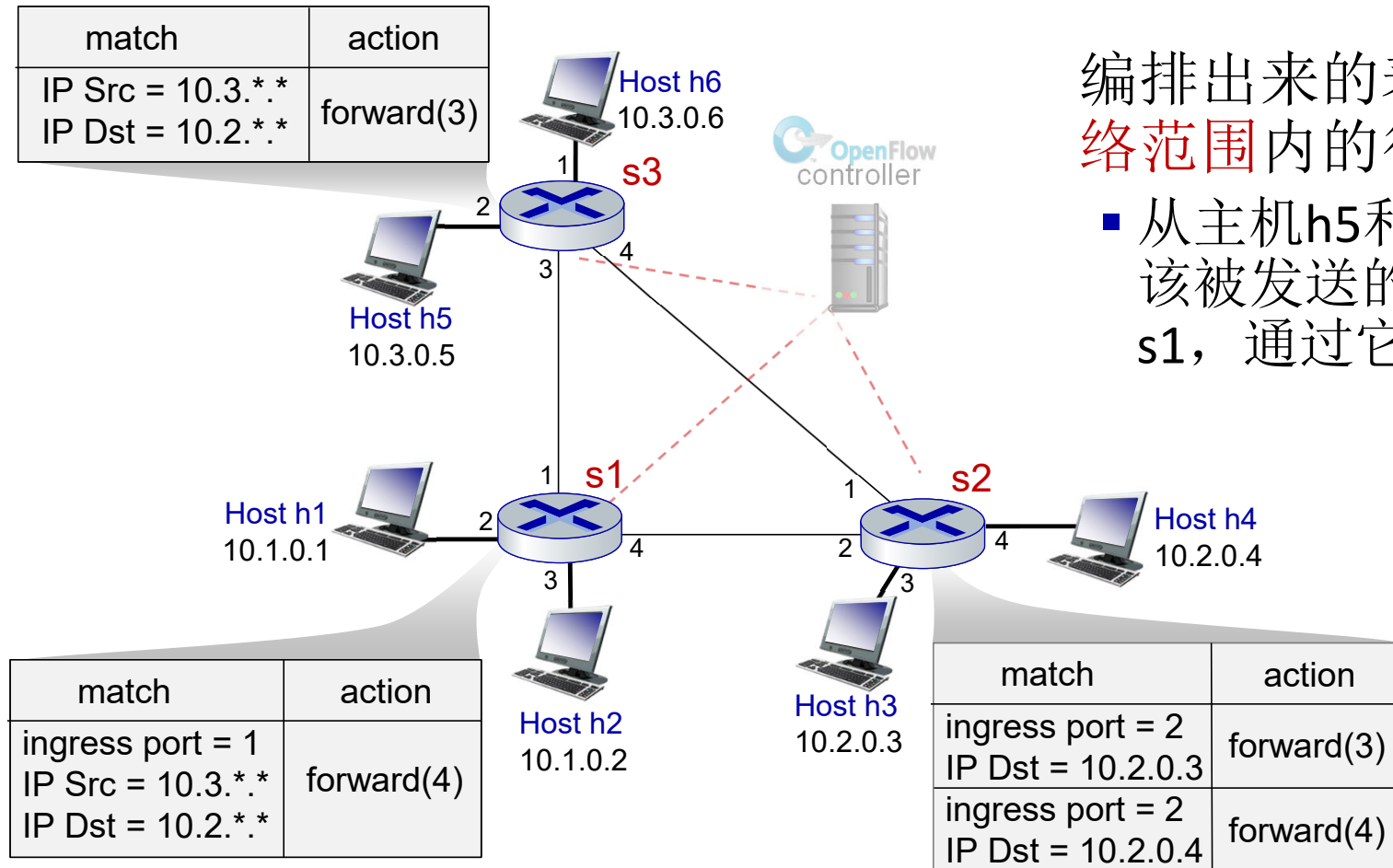
OpenFlow 例子



编排出来的表项可以控制网络范围内的行为, e.g.,:

- 从主机h5和h6来的数据报应该被发送的h3或者h4，通过s1，通过它到达到s2

OpenFlow 例子



编排出来的表项可以控制网络范围内的行为, e.g.,:

- 从主机h5和h6来的数据报应该被发送的h3或者h4, 通过s1, 通过它到达到s2

通用转发：总结

- “匹配+行动”抽象: 将到来分组的（任意层次）头部若干bit与表项进行匹配，然后采取相应的行动
 - 匹配很多字段（链路层，网络层，传输层头部字段）
 - 本地行动：丢弃，转发，修改，或发送匹配的分组到控制器
 - “可编程”的网络范围的行为
- “网络可编程”的简单形式
 - 可编程，每分组“处理”
 - 历史来源：主动网络
 - 目前：更多通用化的编程： P4 (看 p4.org).

网络层：“数据平面”提纲

- 网络层：概述
- 路由器组成结构和工作原理
- IP: the Internet Protocol
- 通用转发
- 中间盒
 - 中间盒功能
 - 互联网演化，互联网体系架构原则



中间盒

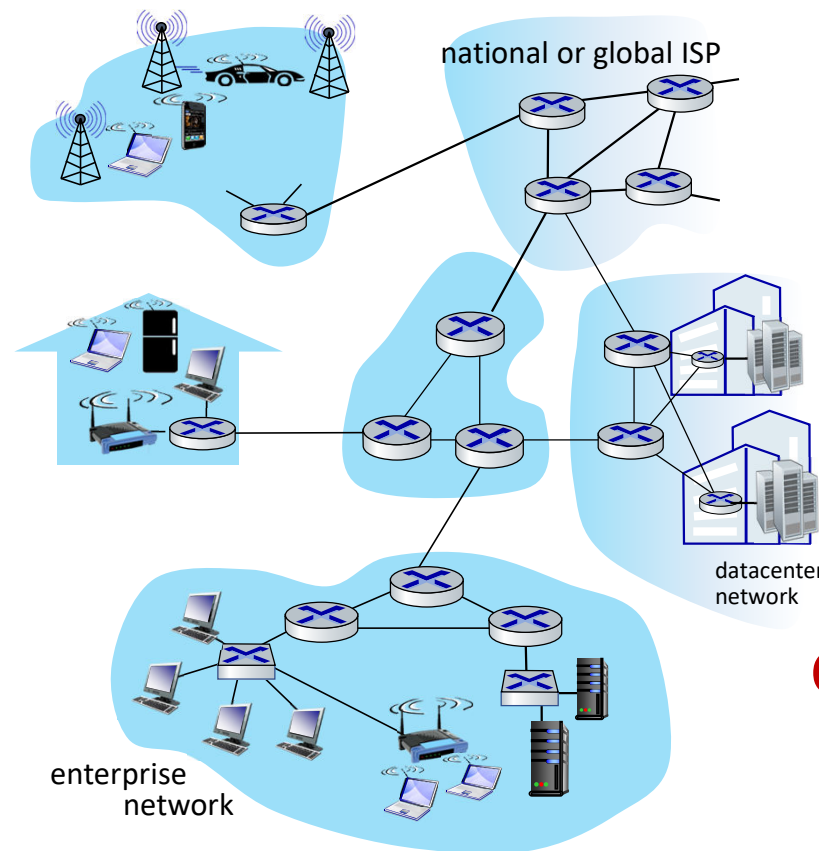
中间盒 (RFC 3234)

“在源主机到目标主机的数据传输路径上，任何一个中间设备如果执行了IP路由器正常、标准功能之外的任何设备，就叫做中间盒”

到处都是中间盒！

NAT: 家庭，蜂窝网络和机构

Application-specific: 服务提供商，机构，CDN



Firewalls, IDS: 公司，机构和服务提供商

Load balancers: 机构服务提供商，数据中心，移动网络

Caches: 服务提供商，移动网络,CDN

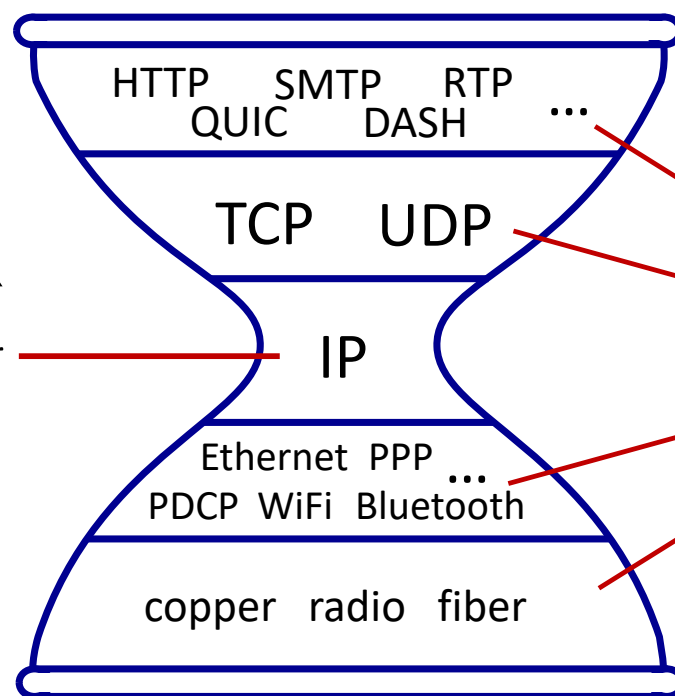
中间盒

- 一开始：专有（封闭）的硬件解决方案
- 转移到实现了开放API的“白盒”硬件
 - 放弃了专有硬件解决方案
 - 通过匹配+行动实现可编程的本地行动
 - 软件走向创新/差异化
- SDN: (逻辑地) 集中化控制和配置管理，通常在私有/公有云中
- network functions virtualization (NFV): 基于白盒组网，计算和存储实现可编程的服务

IP沙漏型网络体系架构

互联网的“瘦腰”：

- 一个网络层协议：IP
- 必须在每一个（数十亿）互联网连接的设备上被实现

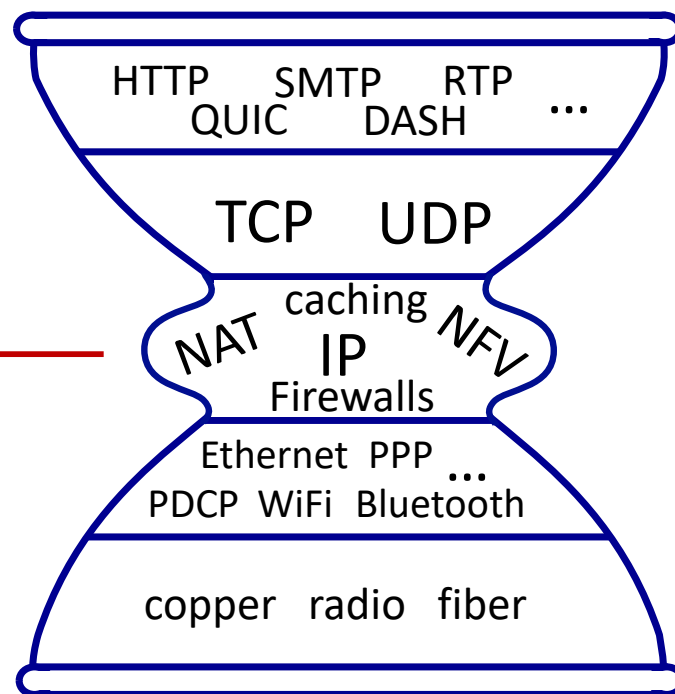


很多在物理层，链路层，传输层和应用层的协议

步入中年的沙漏型IP模型

Internet's middle age
“love handles”?

- 中间盒，在网内进行
操作



互联网体系结构的设计原则

RFC 1958

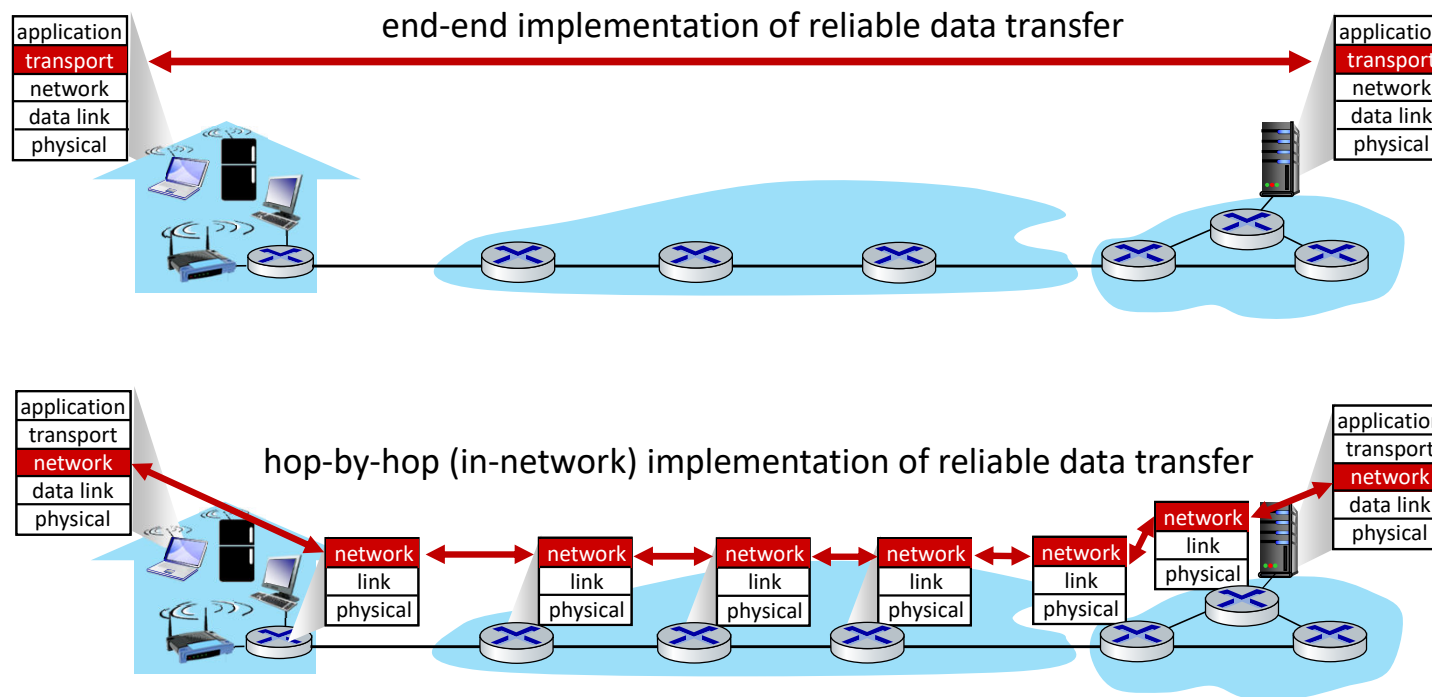
“互联网社区的许多成员会争辩说：没有架构，只有一种传统，在最初的25年里没有被写下来（或者至少没有被IAB写下来）。不过，一般来说，社区成员都会认为认为 “目标是连接性，手段是IP协议，智能是端到端的，而不是隐藏在网络中间的”

3个基本理念：

- 连接的简单性
- IP协议：瘦腰
- 智能性和复杂性在网络边缘

端到端原则

- 一些网络的功能性（例如：可靠数据传输，拥塞控制）可以在网络内部，或者而在网络边缘实现



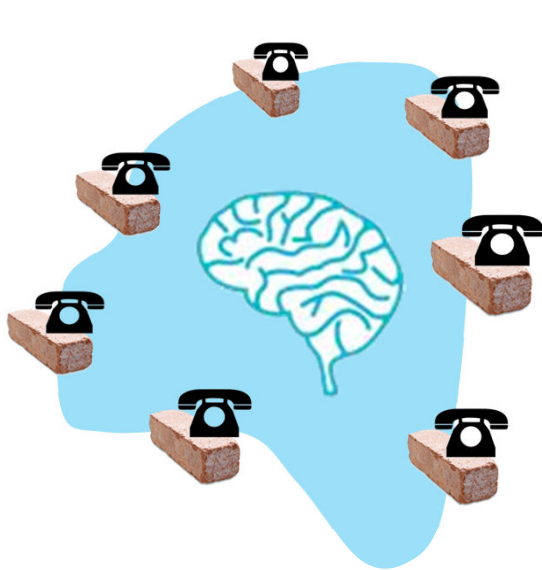
端到端原则

- 一些网络的功能性（例如：可靠数据传输，拥塞控制）可以在网络内部，或者而在网络边缘实现

“只有在通信系统端点应用程序的知识和帮助下，才能完全和正确地实现所讨论的功能，只有在通信系统的端节点处之应用的知识和帮助下，问题中的功能才可以被完整地 and 正确地被实现. 因此，假设被问及的功能作为通信系统本身的特性是不可能的 (有时，通信系统提供的功能的不完整版本可能有助于提高性能.)

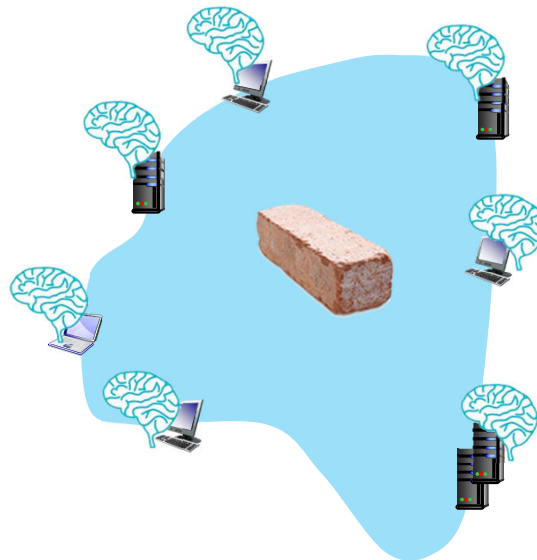
我们把这行有关于反对在低层次实现网络功能的论证叫做“端到端原则”

智能体现在哪里？



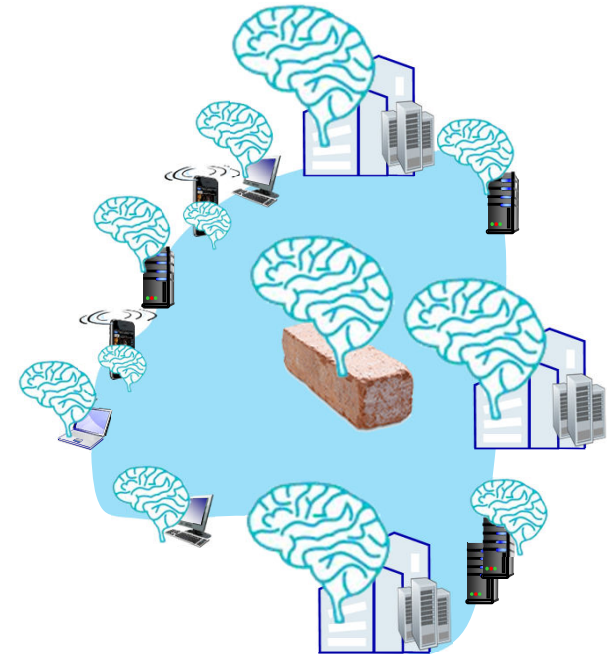
20th 世纪的电话网络：

- 智能/计算体现在网络交换机上



Internet (pre-2005)

- 智能，计算体现在边缘



Internet (post-2005)

- 可编程的网络设备
- 智能，计算，海量的应用层的基础设施在边缘

第4章： 结束！

- 网络层： 概述
- 路由器的组成结构和工作原理
- IP: the Internet Protocol
- 通用转发， SDN
- 中间盒



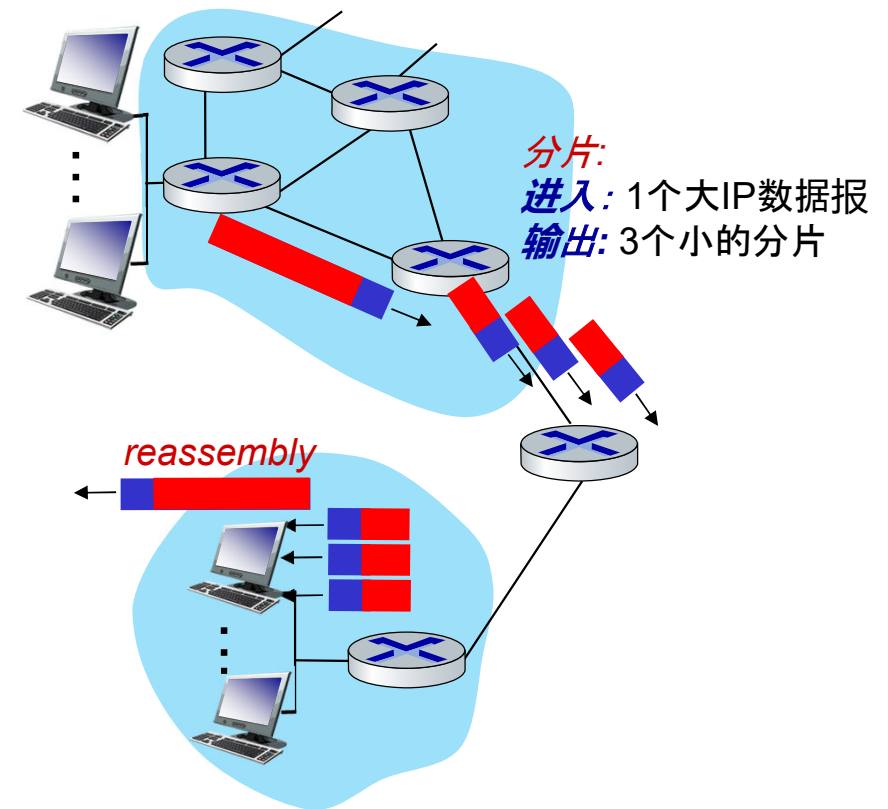
问题: 转发表（基于目标的转发）或者流表（通用转发）是如何计算的？

回答: 由控制平面（下一章）得到

第4章附加幻灯片

IP 分片/重组

- 网络中的链路有MTU（最大传输单元）-最大的、可能的链路层的帧
 - 不同链路类型有不同的MTU
- 一个大的IP数据报在网络内部被切分（“分片”）
 - 一个数据报变成若干数据报
 - 每个数据报都是独立的完整的数据报
 - 在目标端进行“重组”
 - IP头部的若干bit被用于标识，排序这些相关的分组分片



IP 分片/重组

例子:

- 4000 byte 数据报
- MTU = 1500 bytes

	length	ID	fragflag	offset	
	=4000	=x	=0	=0	

一个大的IP数据报变成了若干小的数据报

每个帧有1480 bytes在其
数据载荷部分

偏移量 =
 $1480/8$

	length	ID	fragflag	offset	
	=1500	=x	=1	=0	

	length	ID	fragflag	offset	
	=1500	=x	=1	=185	

	length	ID	fragflag	offset	
	=1040	=x	=0	=370	

DHCP: Wireshark output (home LAN)

Message type: **Boot Request (1)**

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

request

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 0.0.0.0 (0.0.0.0)

Your (client) IP address: 0.0.0.0 (0.0.0.0)

Next server IP address: 0.0.0.0 (0.0.0.0)

Relay agent IP address: 0.0.0.0 (0.0.0.0)

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Server host name not given

Boot file name not given

Magic cookie: (OK)

Option: (t=53,l=1) **DHCP Message Type = DHCP Request**

Option: (61) Client identifier

Length: 7; Value: 010016D323688A;

Hardware type: Ethernet

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Option: (t=50,l=4) Requested IP Address = 192.168.1.101

Option: (t=12,l=5) Host Name = "nomad"

Option: (55) Parameter Request List

Length: 11; Value: 010F03062C2E2F1F21F92B

1 = Subnet Mask; 15 = Domain Name

3 = Router; 6 = Domain Name Server

44 = NetBIOS over TCP/IP Name Server

.....

Message type: **Boot Reply (2)**

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

reply

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 192.168.1.101 (192.168.1.101)

Your (client) IP address: 0.0.0.0 (0.0.0.0)

Next server IP address: 192.168.1.1 (192.168.1.1)

Relay agent IP address: 0.0.0.0 (0.0.0.0)

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Server host name not given

Boot file name not given

Magic cookie: (OK)

Option: (t=53,l=1) DHCP Message Type = DHCP ACK

Option: (t=54,l=4) Server Identifier = 192.168.1.1

Option: (t=1,l=4) Subnet Mask = 255.255.255.0

Option: (t=3,l=4) Router = 192.168.1.1

Option: (6) Domain Name Server

Length: 12; Value: 445747E2445749F244574092;

IP Address: 68.87.71.226;

IP Address: 68.87.73.242;

IP Address: 68.87.64.146

Option: (t=15,l=20) Domain Name = "hsd1.ma.comcast.net."